

Федеральное государственное образовательное бюджетное учреждение
высшего образования
«Финансовый университет при Правительстве Российской Федерации»
(Финансовый университет)

Факультет информационных технологий и анализа больших данных
Кафедра искусственного интеллекта

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

на тему:

**«Разработка фреймворка генерации бенчмарков систем выявления
концептуального дрейфа с автоматизированным формированием и
параметризацией сценариев»**

Направление подготовки 01.03.02 «Прикладная математика и информатика»
Направленность (профиль): [УТОЧНИТЬ У РУКОВОДИТЕЛЯ]

Выполнили:

Будаев Константин Владимирович, студент группы ПМ23-5

Гарифзянов Тимур Русланович, студент группы ПМ23-5

(подписи) _____

Руководитель:

ассистент кафедры искусственного интеллекта

Окунева Эвелина Александровна

(подпись) _____

ВКР соответствует предъявляемым требованиям

Заведующий кафедрой [УТОЧНИТЬ У РУКОВОДИТЕЛЯ]

_____ «___» _____ 2027 г.

Москва — 2027

РЕФЕРАТ

Пояснительная записка содержит 97 с., 19 рис., 34 табл., 60 источн., 4 прил.

КОНЦЕПТУАЛЬНЫЙ ДРЕЙФ, ПОТОК ДАННЫХ, ДЕТЕКТОР ДРЕЙФА, БЕНЧМАРК, СЦЕНАРИЙ ДРЕЙФА, РАССТОЯНИЕ ПОЛНОЙ ВАРИАЦИИ, ПЛАНИРОВАНИЕ ВЫЧИСЛИТЕЛЬНОГО ЭКСПЕРИМЕНТА, ПОСЛЕДОВАТЕЛЬНОСТЬ СОБОЛЯ, СТАТИСТИЧЕСКОЕ СРАВНЕНИЕ АЛГОРИТМОВ, RIVER

Объект исследования — методы выявления концептуального дрейфа в потоках данных. Предмет исследования — методы и программные средства формирования параметризованных сценариев дрейфа для сравнительной оценки детекторов.

Цель работы — разработать фреймворк генерации бенчмарков систем выявления концептуального дрейфа, в котором сценарий дрейфа задаётся формализованным набором параметров, а наборы сценариев формируются автоматически.

В работе построена формальная модель сценария дрейфа с 14 ограничениями допустимости и единой шкалой величины дрейфа — расстоянием полной вариации между совместными распределениями до и после события. Для калиброванного семейства потоков получены точные выражения величины реального, виртуального и априорного дрейфа и ограничения её достижимости. Предложен метод автоматического формирования наборов сценариев по планам вычислительного эксперимента (сетка, случайная выборка, латинский гиперкуб, последовательности Соболя) с тремя способами учёта ограничений, шаблонами с наследованием, композицией и шкалой сложности. Реализован фреймворк scendrift на языке Python: генерация потоков с истинной разметкой дрейфа, формирование и версионирование наборов, прогон детекторов библиотеки river, метрики обнаружения и статистическое сравнение по Демшару.

Экспериментально подтверждено, что генератор воспроизводит заданную величину дрейфа в пределах выборочной погрешности. На автоматически сформированном наборе из 64 сценариев средний F1 девяти детекторов составил от 0,10 до 0,36, а периодический сброс модели без анализа данных получил 0,38. Показано, что главный фактор сложности обнаружения — величина дрейфа на единой шкале, что виртуальный дрейф по потоку ошибок на синтетических данных не обнаруживается и что ручной набор классических потоков завышает качество детекторов и даёт иное ранжирование.

Результаты применимы для выбора детектора дрейфа под характеристики прикладной задачи, для воспроизводимой проверки новых детекторов и в учебном процессе по анализу потоковых данных.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....	6
ВВЕДЕНИЕ.....	8
1 Анализ предметной области и постановка задачи.....	13
1.1 Потоки данных и концептуальный дрейф.....	13
1.2 Таксономия концептуального дрейфа.....	14
1.3 Методы обнаружения дрейфа.....	15
1.4 Бенчмарки, генераторы потоков и средства оценки детекторов.....	18
1.5 Анализ пробелов и постановка задачи.....	20
1.6 Выводы по главе 1.....	21
2 Формальная модель сценария и метод формирования наборов.....	23
2.1 Поток данных и концептуальный дрейф.....	23
2.2 Определение сценария.....	23
2.3 Единая шкала величины дрейфа.....	25
2.4 Пространство параметров и таксономия сценариев.....	29
2.5 Ограничения допустимости и истинная разметка.....	31
2.6 Протокол оценки и метрики.....	32
2.7 Метод формирования наборов сценариев.....	35
2.8 Выводы по главе 2.....	38
3 Проектирование и реализация фреймворка.....	40
3.1 Требования к фреймворку.....	40
3.2 Архитектура.....	41
3.3 Декларативное описание и идентификация сценариев.....	44
3.4 Генерация потоков.....	45
3.5 Формирование наборов сценариев.....	47
3.6 Оценка детекторов.....	48
3.7 Тестирование и воспроизводимость.....	52
3.8 Выводы по главе 3.....	53
4 Экспериментальное исследование.....	55
4.1 План экспериментов.....	55

4.2 Э1. Валидация генератора.....	56
4.3 Сравнение планов и способов учёта ограничений.....	60
4.4 Э2. Основной бенчмарк.....	64
4.5 Э3. Чувствительность и области отказа.....	66
4.6 Э4. Шкала сложности.....	70
4.7 Э5. Статистическое сравнение.....	73
4.8 Э6. Сравнение с ручным набором классических потоков.....	75
4.9 Э7. Реальные данные с внедрённым дрейфом.....	77
4.10 Э8. Влияние на ассигасу.....	78
4.11 Обсуждение и угрозы валидности.....	79
4.12 Выводы по главе 4.....	80
ЗАКЛЮЧЕНИЕ.....	82
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	85
ПРИЛОЖЕНИЕ А Каталог ограничений допустимости сценария.....	93
ПРИЛОЖЕНИЕ Б Примеры описания сценариев и конфигурации набора....	94
ПРИЛОЖЕНИЕ В Распределение работ между исполнителями.....	96
ПРИЛОЖЕНИЕ Г Структура программного комплекса и воспроизведение результатов.....	97

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В пояснительной записке применяют следующие сокращения и обозначения.

ADWIN — ADaptive WINdowing, детектор дрейфа с адаптивным окном.

CD — critical difference, критическая разность средних рангов в критерии Немецьи; в п. 2.7 и 4.3 — центрированное расхождение плана (centered discrepancy).

DDM, EDDM — Drift Detection Method и Early Drift Detection Method.

FAR — false alarm rate, число ложных тревог на 1000 объектов.

FHDDM — Fast Hoeffding Drift Detection Method.

HDDM_A, HDDM_W — детекторы на основе неравенства Хёфдинга со средними и со взвешенными средними.

KSWIN — Kolmogorov–Smirnov WINdowing, детектор на основе критерия Колмогорова–Смирнова.

LHS — Latin hypercube sampling, латинский гиперкуб.

MDR — missed detection rate, доля пропущенных событий дрейфа.

MTD — mean time to detection, средняя задержка обнаружения.

MTFA — mean time between false alarms, среднее время между ложными тревогами.

MTR — mean time ratio, сводное отношение MTFA, MTD и MDR.

PageHinkley, PageHinkley-up — детектор Пейджа–Хинкли в двустороннем и в одностороннем вариантах.

TV — total variation, расстояние полной вариации.

ГСЧ — генератор псевдослучайных чисел.

КС — критерий Колмогорова–Смирнова и его статистика.

$S = \langle B, E, \Omega, s \rangle$ — сценарий дрейфа: базовая конфигурация потока, список событий, протокол оценки, seed реализации.

κ , φ , m , ℓ , α — вид, форма, величина, ширина перехода и доля затронутых признаков события дрейфа.

π_0, η, d, n — доля положительного класса, вероятность инверсии метки, размерность и длина потока.

W, Δ — разогрев и окно допуска протокола оценки.

λ — параметр шкалы сложности.

ВВЕДЕНИЕ

Модели машинного обучения в эксплуатации работают с потоками данных, свойства которых меняются со временем: меняется поведение клиентов банка, появляются новые схемы мошенничества, меняются правила и цены на рынках. Изменение совместного распределения признаков и целевой переменной $P(X, y)$ во времени называют концептуальным дрейфом (concept drift) [1; 2]. Без своевременного обнаружения дрейфа качество модели снижается незаметно для её владельца, поэтому детекторы дрейфа стали обязательной частью систем мониторинга моделей.

Детекторов предложено много: DDM, EDDM, ADWIN, KSWIN, Page–Hinkley, семейство HDDM, FHDDM и другие. Их сравнение при этом по-прежнему опирается на ограниченный набор вручную составленных потоков (SEA, Agrawal, STAGGER, Hyperplane и др.) с несколькими фиксированными конфигурациями. Известно, что выводы такого сравнения существенно зависят от свойств тестовых потоков и от протокола оценки [3; 4]. Вручную составленный набор не позволяет систематически ответить на вопросы, важные для практики: при какой величине и скорости дрейфа детектор надёжно его обнаруживает, как на качество влияют шум меток, дисбаланс классов и доля затронутых признаков, где проходят границы области отказа детектора.

Для ответа на эти вопросы нужен инструмент, который формирует сценарии дрейфа автоматически, причём каждый параметр сценария имеет проверяемый смысл: заявленная величина дрейфа действительно реализуется в данных, недостижимые сочетания параметров обнаруживаются до генерации, а набор сценариев равномерно покрывает пространство параметров. Существующие средства (MOA, river, CapuMOA, stream-learn и др.) решают эти задачи частично; подробный анализ приведён в главе 1. Этим определяется актуальность работы.

Цель работы — разработать фреймворк генерации бенчмарков систем выявления концептуального дрейфа, в котором сценарий дрейфа задаётся

формализованным набором параметров, а наборы сценариев формируются автоматически.

Для достижения цели поставлены задачи:

1) Проанализировать виды концептуального дрейфа, методы его обнаружения, существующие бенчмарки и генераторы потоков, выявить их ограничения.

2) Построить формальную модель сценария дрейфа: пространство параметров, единую шкалу величины дрейфа, ограничения допустимости.

3) Спроектировать архитектуру фреймворка и декларативный формат описания сценариев.

4) Реализовать генераторы потоков с контролируемым дрейфом и автоматическим построением истинной разметки дрейфа.

5) Реализовать модуль автоматического формирования и параметризации наборов сценариев.

6) Реализовать единый интерфейс детекторов, раннер бенчмарка, модуль метрик и статистического сравнения, формирование отчётов.

7) Провести экспериментальное исследование: проверить генератор, сравнить детекторы на сформированных наборах, проанализировать чувствительность к параметрам сценария.

Объект исследования — методы выявления концептуального дрейфа в потоках данных.

Предмет исследования — методы и программные средства формирования параметризованных сценариев дрейфа для сравнительной оценки детекторов.

Методы исследования: теория вероятностей и математическая статистика (расстояние полной вариации, многомерное нормальное распределение), планирование вычислительного эксперимента (латинский гиперкуб, последовательности с малым расхождением), метод Монте-Карло, непараметрические критерии сравнения алгоритмов (Фридмана, Неменьи,

Уилкоксона), объектно-ориентированное проектирование и тестирование программного обеспечения.

Научная новизна работы состоит в следующем:

1) Предложена формальная модель сценария дрейфа — типизированный кортеж $S = \langle B, E, \Omega, s \rangle$ с явной системой ограничений допустимости и с разделением структурной случайности (геометрия концептов) и случайности реализации (выборка объектов и шум). Известные таксономии дрейфа [5; 6] описательны, а генераторы MOA, river, СаруМОА и stream-learn задают параметры разрозненно; предложенная модель позволяет проверять сценарии автоматически и воспроизводимо их порождать.

2) Введена единая шкала величины дрейфа — расстояние полной вариации между совместными распределениями $P(X, y)$ до и после события — и получена её аналитическая калибровка для трёх видов дрейфа в семействе «гиперплоскость над гауссовыми признаками», включая дисбаланс классов и поворот в части признаков. Из калибровки выведены ограничения достижимости величины. В известных генераторах с управляемой величиной [7; 8] величина подбирается итерационно и только для одного вида изменения.

3) Разработан метод автоматического формирования наборов сценариев по планам вычислительного эксперимента с учётом ограничений допустимости. Предложены три способа учёта ограничений, в том числе условные домены, отображающие план в допустимую область без потери точек и без скопления на её границе; их свойства измерены. В проанализированных работах наборы сценариев формируются полным перебором [9] или случайной выборкой [10].

4) Предложена методика описания областей отказа детекторов в пространстве параметров сценария (тепловые карты, корреляционный анализ и правила дерева решений) с поправкой на случайный уровень обнаружения, вызванный ложными тревогами.

Заимствованные элементы указаны в тексте со ссылками на источники: детекторы и базовая модель взяты из библиотеки river [11], понятие severity — из работы Minku и др. [6], метрики MTD, MDR, MTFA и MTR — из CD-MOA [12], методика статистического сравнения — по Демшару [13], генераторы латинского гиперкуба и последовательностей Соболя — из библиотеки SciPy [14].

Практическая значимость работы: фреймворк позволяет выбирать детектор под характеристики конкретной задачи (величина и скорость дрейфа, дисбаланс, шум), проверять новые детекторы на воспроизводимых наборах сценариев и кэшировать результаты благодаря идентификации сценариев по содержимому. Эксперименты выявили особенности протокола оценки (влияние сброса модели, «иллюзию прогресса» периодического сброса, расширение окон допуска), которые следует учитывать в любом сравнении детекторов.

Работа выполнена коллективом из двух исполнителей. Будаев К. В. отвечал за сценарии и данные: формальную модель сценария, шкалу величины дрейфа, ограничения допустимости, генерацию потоков, формирование наборов сценариев, эксперименты по валидации генератора и на реальных данных. Гарифзянов Т. Р. отвечал за оценку: протокол оценки и метрики, адаптеры детекторов, раннер, статистическое сравнение, отчёты и основную часть экспериментов с детекторами. Обзор аналогов, архитектура, шкала сложности и выводы выполнены совместно. Подробное распределение работ по разделам записки и модулям фреймворка приведено в приложении В.

Пояснительная записка состоит из введения, четырёх глав, заключения, списка использованных источников и приложений. В главе 1 проанализированы предметная область, существующие средства и сформулирована постановка задачи. В главе 2 построена формальная модель сценария и описан метод формирования наборов сценариев. В главе 3

приведены проектирование и реализация фреймворка. В главе 4 изложены результаты экспериментального исследования.

1 Анализ предметной области и постановка задачи

1.1 Потоки данных и концептуальный дрейф

Поток данных (data stream) — потенциально неограниченная последовательность объектов, поступающих во времени [1; 15]. Модель, работающая на потоке, должна обрабатывать каждый объект один раз, укладываться в ограниченную память, выдавать прогноз в любой момент и адаптироваться к изменениям. Последнее требование связано с концептуальным дрейфом — изменением распределения $P_t(X, y)$ с течением времени [1; 2; 16].

Причины дрейфа в прикладных задачах разнообразны:

- изменение поведения клиентов (сезонность, кризисы, новые продукты);
- адаптация злоумышленников к системам противодействия мошенничеству;
- смена регламентов и рыночных условий.

Классический пример — набор Electricity (Elec2) с ценами на электроэнергию в Новом Южном Уэльсе, где правила рынка менялись со временем [17].

Работа с дрейфом включает две задачи [1; 2]: обнаружение (detection) — установить, что распределение изменилось, и указать момент изменения, и адаптацию (adaptation) — обновить модель. Настоящая работа относится к первой задаче, точнее к оценке методов её решения.

Детекторы дрейфа делятся на две группы. Детекторы с учителем отслеживают поток ошибок классификатора и поэтому нуждаются в истинных метках. Детекторы без учителя отслеживают распределение признаков [18; 19]. Основное внимание в работе уделено детекторам с учителем: именно они реализованы в библиотеке river и чаще всего сравниваются в литературе [3; 20].

1.2 Таксономия концептуального дрейфа

Дрейф классифицируют по нескольким независимым признакам [1; 5; 6].

По источнику изменения [1; 21]. Совместное распределение раскладывается двумя способами: $P(X, y) = P(X)P(y|X) = P(y)P(X|y)$. В зависимости от того, какая компонента меняется, различают:

- реальный дрейф (real concept drift, concept shift) — меняется $P(y|X)$: при тех же признаках правильный ответ другой. Такой дрейф, как правило, ухудшает модель, обученную на старом концепте;

- виртуальный дрейф (virtual drift, covariate shift) — меняется $P(X)$ при неизменном $P(y|X)$. Модель может не пострадать, но обнаруживать такой дрейф всё равно важно: модель работает на непривычных данных;

- дрейф априорных вероятностей (prior probability shift, label shift) — меняется $P(y)$ при неизменном $P(X|y)$. Типичный пример — рост доли мошеннических операций.

По форме во времени [1; 5] различают внезапный дрейф (sudden, abrupt), при котором концепт сменяется мгновенно; постепенный (gradual), при котором какое-то время объекты приходят то из старого, то из нового концепта и доля нового растёт; инкрементальный (incremental), при котором концепт меняется непрерывно через последовательность промежуточных концептов; повторяющийся (recurring), при котором поток возвращается к ранее встречавшемуся концепту. Формы дрейфа иллюстрирует рисунок 1.1.

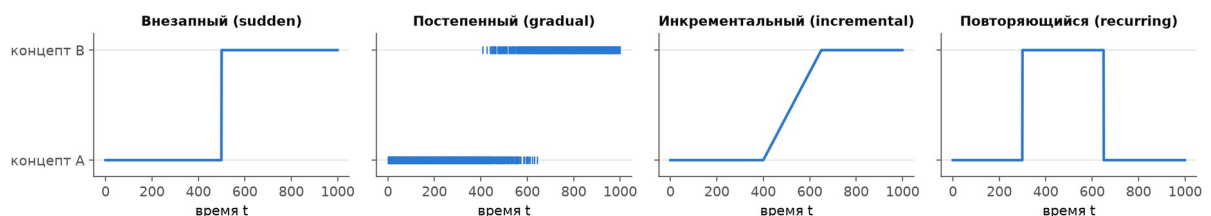


Рисунок 1.1 — Формы концептуального дрейфа во времени

По величине и скорости. Minku, White и Yao [6] ввели характеристику severity — объём изменений, вызванных новым концептом, который

измеряется, в частности, долей входного пространства, где меняется класс, и характеристику *speed* — величину, обратную времени полного замещения старого концепта новым. Webb и соавторы [5] формализовали *magnitude* как расстояние между распределениями и *duration* как длительность дрейфа.

По охвату дрейф бывает глобальным, если затрагивает всё пространство признаков, и локальным, если затрагивает его часть, например подмножество признаков или область пространства.

Модель сценария, построенная в главе 2, задаёт эти оси как независимые параметры: вид κ отвечает источнику изменения, форма φ и ссылка возврата ρ — форме во времени, величина m (расстояние полной вариации между концептами) обобщает *severity*, ширина перехода ℓ — длительность, обратная скорости, доля затронутых признаков α — охват.

1.3 Методы обнаружения дрейфа

Детекторы с учителем получают поток индикаторов ошибки $e_t = 1[\hat{y}_t \neq y_t]$ базового классификатора и сигнализируют о существенном изменении частоты ошибок [1; 2]. Направленность реакции у них разная: DDM, EDDM, HDDM_A, HDDM_W и FHDDM реагируют только на рост ошибки, а ADWIN, KSWIN и Page–Hinkley в *river* по умолчанию — на изменение в обе стороны. В работе сравниваются девять детекторов: восемь алгоритмов, причём Page–Hinkley используется в двустороннем варианте *river* по умолчанию и в одностороннем (далее PageHinkley-up). Детекторы относятся к трём группам.

Первая группа — статистический контроль частоты ошибок:

– DDM (Drift Detection Method) [22] моделирует число ошибок биномиальным законом и отслеживает минимум величины $p_t + s_t$, где p_t — частота ошибок, $s_t = \sqrt{p_t(1 - p_t)/t}$. Предупреждение выдаётся при $p_t + s_t \geq p_{min} + 2s_{min}$, дрейф — при $p_t + s_t \geq p_{min} + 3s_{min}$;

– EDDM (Early DDM) [23] отслеживает расстояние между соседними ошибками, что повышает чувствительность к медленному постепенному дрейфу;

– HDDM_A и HDDM_W [24] сравнивают оценки средней ошибки до и после возможной точки изменения с помощью границ типа Хёффдинга; вариант A использует обычные средние, вариант W — экспоненциально взвешенные. Предположений о виде распределения нет, нужна только ограниченность значений;

– FHDDM (Fast Hoeffding Drift Detection Method) [25] хранит скользящее окно из n индикаторов верного ответа и отслеживает максимум доли верных ответов в окне. Дрейф фиксируется, когда доля падает ниже максимума больше чем на $\varepsilon = \sqrt{\ln(1/\delta)/(2n)}$.

Вторая группа — адаптивные окна и сравнение распределений:

– ADWIN (ADaptive WINdowing) [26] хранит окно переменной длины. Если найдено разбиение окна на две части со статистически различными средними (граница Хёффдинга с уровнем δ), устаревшая часть отбрасывается. Для ADWIN доказаны оценки вероятности ложного срабатывания и пропуска изменения;

– KSWIN (Kolmogorov–Smirnov WINdowing) [27] сравнивает критерием Колмогорова–Смирнова последние значения окна со случайной выборкой из более ранней части окна.

Третья группа — последовательный анализ. Детектор Page–Hinkley [28] — вариант метода кумулятивных сумм: накапливается сумма $M_T = \sum_{t \leq T} (x_t - \hat{x}_t - \delta)$, и дрейф фиксируется, когда $M_T - \min_{t \leq T} M_t > \lambda$. В river суммы затухают с коэффициентом забывания, а по умолчанию проверяется изменение в обе стороны.

В основном эксперименте используются гиперпараметры river 0.26.1 по умолчанию [11]; они приведены в таблице 1.1.

Таблица 1.1 — Детекторы и их гиперпараметры по умолчанию в river 0.26.1

Детектор	Группа	Вход	Гиперпараметры по умолчанию
ADWIN	окно	вещественный	delta = 0,002; clock = 32; grace_period = 10
KSWIN	окно	вещественный	alpha = 0,005; window_size = 100; stat_size = 30
PageHinkley	последовательный анализ	вещественный	min_instances = 30; delta = 0,005; threshold = 50; alpha = 0,9999; mode = both
PageHinkley-up	последовательный анализ	вещественный	то же, mode = up
DDM	контроль ошибок	бинарный	warm_start = 30; warning_threshold = 2; drift_threshold = 3
EDDM	контроль ошибок	бинарный	warm_start = 30; alpha = 0,95; beta = 0,9
HDDM_A	контроль ошибок	бинарный	drift_confidence = 0,001; warning_confidence = 0,005
HDDM_W	контроль ошибок	бинарный	drift_confidence = 0,001; warning_confidence = 0,005; lambda_val = 0,05
FHDDM	контроль ошибок	бинарный (верный ответ)	sliding_window_size = 100; confidence_level = 0,000001

Известно, что результаты сравнения детекторов существенно зависят от свойств тестовых потоков и от протокола оценки [3; 4; 20]. Barros и Santos сравнили 14 детекторов на большом числе вручную сконфигурированных синтетических потоков [3]. Vifet показал, что популярный протокол «ассурасу классификатора, перезапускаемого при каждом срабатывании» может создавать «иллюзию прогресса»: даже детектор, не обнаруживающий изменений, способен повышать ассурасу из-за временной зависимости данных [4]. Для оценки качества обнаружения относительно истинных моментов изменения в CD-МОА предложены метрики [12]: MTFA — среднее время между ложными тревогами, MTD — среднее время до обнаружения, MDR — доля пропущенных изменений, MTR — их сводное отношение.

Эти наблюдения определяют главное требование к бенчмарку: он должен систематически варьировать свойства сценария и знать истинную разметку дрейфа.

1.4 Бенчмарки, генераторы потоков и средства оценки детекторов

Рассмотрим программные средства, применяемые для генерации потоков с дрейфом и оценки детекторов.

MOA [15; 29] — эталонная платформа на языке Java. В неё входят генераторы SEA [30], Agrawal [31], STAGGER [32], вращающейся гиперплоскости [33] и другие; обёртка ConceptDriftStream с сигмоидным переходом, которую можно вкладывать саму в себя; 15 детекторов; модуль CD-MOA с метриками MTFa, MTD, MDR и MTR [12]; экспериментатор со статистическими тестами.

Библиотека scikit-multiflow [34] перенесла генераторы и детекторы MOA на Python и в 2020 г. вошла в river [11]. River — основная Python-библиотека для потоков: 19 синтетических генераторов, ConceptDriftStream, детекторы ADWIN, DDM, EDDM, FHDDM, HDDM_A, HDDM_W, KSWIN и Page–Hinkley с единым интерфейсом. Метрик качества обнаружения, экспорта истинной разметки и описания сценариев в river нет.

CaruMOA [35] — Python-обёртка над MOA. Сценарий в ней собирается декларативно из чередующихся концептов и переходов, моменты дрейфа выдаются отдельным методом, а оценщик детекторов считает precision, recall, F1, задержку и частоту ложных тревог.

В библиотеке stream-learn [36] генератор потоков одним вызовом задаёт число дрейфов, их резкость, инкрементальность и повторяемость, шум меток и дисбаланс, в том числе динамический. Однако позиции дрейфов всегда равноотстоящие, а величину дрейфа задать нельзя: каждый новый концепт случаен.

Frouros [37] и Menelaus [38] — библиотеки детекторов. В Frouros генерации сценариев почти нет. В Menelaus есть операторы внедрения ковариатного, реального и априорного дрейфа в реальные данные на заданном окне.

Генератор Петижана–Уэбба для MOA [7] к работе [5] порождает категориальные потоки с внезапным дрейфом заданной величины

(расстояние Хеллингера) отдельно для $P(X)$ и $P(y|X)$. Величина достигается итерационным подбором с допуском.

Сравнение средств по признакам, существенным для автоматического бенчмаркинга, приведено в таблице 1.2. Оно составлено по исходному коду и документации актуальных версий на октябрь 2026 г. Признаки обозначены так:

- П1 — поддерживаются формы sudden, gradual, incremental и recurring;
- П2 — вид дрейфа (real, virtual, prior) задаётся явно;
- П3 — величина дрейфа имеет измеримую семантику;
- П4 — истинная разметка дрейфа выдаётся как данные;
- П5 — сценарий описывается декларативно;
- П6 — проверяются ограничения допустимости сценария;
- П7 — наборы сценариев формируются автоматически;
- П8 — вычисляются метрики обнаружения (задержка, ложные тревоги);
- П9 — выполняется статистическое сравнение детекторов.

Таблица 1.2 — Сравнение средств генерации потоков и оценки детекторов («+» — есть, «±» — частично, «–» — нет)

Средство	П1	П2	П3	П4	П5	П6	П7	П8	П9
MOA и CD-MOA	+	±	±	±	±	±	±	±	+
scikit-multiflow	±	–	±	–	–	±	–	–	–
river	±	±	±	±	–	±	–	–	–
CapuMOA	+	±	±	+	±	±	–	+	–
stream-learn	+	±	–	±	–	±	–	±	–
Frouros	±	±	–	–	–	±	–	–	–
Menelaus	±	+	±	±	–	±	–	–	–
генератор Петижана–Уэбба	±	±	+	±	±	±	–	–	–
scendrift (настоящая работа)	+	+	+	+	+	+	+	+	+

Помимо программных средств, существуют исследовательские бенчмарки. Barros и Santos сравнили детекторы на большом числе вручную сконфигурированных искусственных потоков с двумя базовыми классификаторами [3], Gonçalves и др. сравнили восемь детекторов [20].

Aguilar и Cano построили полным перебором 2760 потоков с дрейфом разной локальности и сравнили 9 детекторов [9]. Brzezinski и др. систематически варьировали факторы сложности несбалансированных потоков [39]. Cerqueira и др. предложили каркас оценки детекторов с внедрением в реальные данные изменений четырёх типов со случайными параметрами, метриками с учётом времени и подбором гиперпараметров [10]. Žliobaitė предложила контролируемые перестановки для получения множества тестовых наборов [40]. Зависимость качества обнаружения от свойств изменения изучалась по отдельным параметрам: от размерности при фиксированной величине [41], от доли затронутой подгруппы [42], от величины и доли затронутых данных в пакетной постановке [43]. Souza и др. показали ограничения бенчмарков на реальных данных: истинные моменты дрейфа в них неизвестны [44].

1.5 Анализ пробелов и постановка задачи

Из обзора следует пять пробелов.

Первый пробел — нет общей семантики величины дрейфа. У большинства генераторов «величина» — внутренний параметр, несопоставимый между генераторами и видами дрейфа. Калиброванная величина известна для категориальных данных и внезапного дрейфа (генератор Петижана–Уэбба, расстояние Хеллингера) и для ковариатного сдвига (Carrera и Boracchi, дивергенция Кульбака–Лейблера), причём в обоих случаях — с итерационным подбором [7; 8]. Единой шкалы для реального, виртуального и априорного дрейфа с аналитической калибровкой в проанализированных источниках не выявлено.

Второй пробел — сценарий не является проверяемым объектом. В MOA, river и СаруMOA проверяются только структура описания и диапазоны отдельных опций. Недостижимые сочетания параметров (большая величина при малой доле затронутых признаков, конфликт дисбаланса и величины, перекрытие переходов) молча порождают не тот поток, который заявлен.

Третий пробел — наборы сценариев формируются вручную, полным перебором [9] или случайной выборкой [10]. Пространственно-заполняющие планы — латинский гиперкуб [45] и последовательности Соболя [46] — с учётом ограничений допустимости для этой цели в проанализированных источниках не применялись.

Четвёртый пробел — генерация, метрики и статистика разнесены по разным инструментам. CD-МОА даёт метрики, но лишь три одномерных генератора; СаруМОА даёт сценарии и метрики, но не статистику; river даёт детекторы, но не оценку.

Пятый пробел — области отказа детекторов изучаются по одному параметру за раз [41; 42], без описания многопараметрических областей в пространстве сценариев. Методологический прецедент интерпретируемого описания того, где алгоритм работает плохо, есть в анализе пространства экземпляров для статических задач [47].

Постановка задачи. Требуется разработать фреймворк, который:

1) Описывает сценарий дрейфа формальным кортежем параметров с единой калиброванной шкалой величины и проверяемыми ограничениями (пробелы 1 и 2).

2) Автоматически формирует наборы сценариев по планам вычислительного эксперимента с учётом ограничений (пробел 3).

3) В одном конвейере порождает потоки с истинной разметкой дрейфа, прогоняет детекторы, вычисляет метрики обнаружения и выполняет статистическое сравнение (пробел 4).

4) Позволяет описывать области отказа детекторов в пространстве параметров сценария (пробел 5).

1.6 Выводы по главе 1

1) Концептуальный дрейф классифицируется по источнику изменения, форме во времени, величине, скорости и охвату; эти оси независимы и могут служить параметрами сценария.

2) Детекторы с учителем отслеживают поток ошибок базовой модели и различаются направленностью реакции; это существенно для протокола оценки, в котором модель сбрасывается при срабатывании.

3) Существующие средства генерации и оценки решают задачу бенчмаркинга частично: ни одно из них не сочетает измеримую величину дрейфа для всех видов, проверку допустимости сценария, автоматическое формирование наборов и статистическое сравнение.

4) Сформулированы пять пробелов и постановка задачи, которой посвящены главы 2–4.

2 Формальная модель сценария и метод формирования наборов

В главе вводится формальное определение сценария дрейфа: задаются семантика его параметров, единая шкала величины, пространство параметров, таксономия и ограничения допустимости (подразделы 2.1–2.5), протокол оценки и метрики (подраздел 2.6), метод автоматического формирования наборов сценариев (подраздел 2.7). События дрейфа нумеруются с единицы: $e_1, \dots, e_k$; концепт c_0 — исходный, концепт c_k возникает после события e_k .

2.1 Поток данных и концептуальный дрейф

Поток данных — последовательность пар (x_t, y_t) , $t=1, \dots, n$, где $x_t \in R^d$, $y_t \in \{0, 1\}$, а пара (x_t, y_t) порождается распределением $P_t(X, y)$. Концептом называется совместное распределение $P_t(X, y)$ [1], а концептуальным дрейфом между моментами t и u — неравенство $P_t(X, y) \neq P_u(X, y)$.

Используются два разложения совместного распределения:

$$P(X, y) = P(X)P(y|X) = P(y)P(X|y). \quad (2.1)$$

От того, какая компонента разложения (2.1) меняется, зависит вид дрейфа [5; 21]: при реальном дрейфе меняется $P(y|X)$, при виртуальном — только $P(X)$, при дрейфе априорных вероятностей — $P(y)$ при неизменном $P(X|y)$.

2.2 Определение сценария

Определение 1 (сценарий дрейфа). Сценарий — кортеж

$$S = \langle B, E, \Omega, s \rangle, \quad (2.2)$$

где $B = (f, n, d, \pi_0, \eta, c)$ — базовая конфигурация потока: f — семейство генератора, n — длина потока, d — размерность, $\pi_0 \in [0, 0.1; 0, 5]$ — исходная доля положительного (миноритарного) класса, $\eta \in [0; 0, 5]$ — вероятность инверсии метки, c — структурный seed; $E = (e_1, \dots, e_k)$ — упорядоченный

список событий дрейфа; Ω — протокол оценки (подраздел 2.6); s — seed реализации.

Событие дрейфа задаётся кортежем

$$e_k = (\tau_k, \ell_k, \varphi_k, \kappa_k, m_k, \alpha_k, \rho_k), \quad (2.3)$$

где τ_k — центр перехода; ℓ_k — ширина перехода; $\varphi_k \in \{\text{sudden}, \text{gradual}, \text{incremental}\}$ — форма перехода; $\kappa_k \in \{\text{real}, \text{virtual}, \text{prior}\}$ — вид дрейфа; $m_k \in (0; 1]$ — величина; $\alpha_k \in (0; 1]$ — доля затронутых признаков; ρ_k — номер концепта, к которому поток возвращается при повторяющемся дрейфе.

Интервал перехода события — полуинтервал $[onset_k, end_k)$, где

$$onset_k = \tau_k - \lfloor \ell_k / 2 \rfloor, end_k = onset_k + \ell_k. \quad (2.4)$$

Положение объекта внутри перехода задаёт функция перехода $p_k(t)$, растущая от 0 до 1; она бывает линейной или сигмоидной. Формы перехода различаются так:

- внезапный дрейф ($\text{sudden}, \ell_k = 0$) — концепт меняется мгновенно;
- постепенный дрейф (gradual) — объект порождается новым концептом с вероятностью $p_k(t)$, иначе старым;
- инкрементальный дрейф (incremental) — параметры концепта непрерывно меняются от c_{k-1} к c_k : угол поворота нормали равен $p_k(t)\psi_k$, сдвиг — $p_k(t)\delta_k$, доля класса — $(1 - p_k(t))\pi_{k-1} + p_k(t)\pi_k$.

Повторяющийся дрейф не считается отдельной формой. Это событие с заданным ρ_k , то есть возврат к ранее встречавшемуся концепту, а сам возврат может быть внезапным или постепенным. Тем самым виды дрейфа во времени [1] раскладываются на две независимые оси: форму перехода и повторяемость.

Определение 2 (генератор). Генератор семейства f — детерминированное отображение

$$\Gamma_f : S \mapsto (X_{1..n}, y_{1..n}, G), \quad (2.5)$$

где G — истинная разметка дрейфа (ground truth, подраздел 2.5).

В модели два seed вместо одного. Структурный seed s задаёт геометрию концептов: какие признаки затронуты, направления поворота и

сдвига, случайное расписание событий. Seed реализации s задаёт выборку объектов и шум. Отсюда два следствия: повторы сценария с разными s имеют одни и те же концепты, а значит одну и ту же истинную величину дрейфа; проверку допустимости можно выполнить заранее, не порождая данных.

2.3 Единая шкала величины дрейфа

Параметр «величина дрейфа» в существующих генераторах обычно не имеет общей семантики. Это номер варианта функции классификации, на который переключается генератор SEA [30] или Agrawal [31], скорость вращения гиперплоскости [33] или иной внутренний параметр генератора. Поэтому нельзя утверждать, что дрейф величины 0,2 в одном потоке «такой же», как в другом.

Идея измерять величину расстоянием между распределениями не нова. Webb и соавторы определяют magnitude через расстояние Хеллингера [5], а генератор Петижана–Уэбба подбирает категориальные распределения под заданное расстояние Хеллингера [7]. Carrera и Boracchi итерационно калибруют ротансляцию гауссовых данных под заданную дивергенцию Кульбака–Лейблера [8]. В настоящей работе величина определяется одной метрикой для всех трёх видов дрейфа — расстоянием полной вариации между совместными распределениями, — и для калиброванного семейства эта метрика обращается аналитически.

Семейство `hyperplane_gauss`. Концепт k задаётся состоянием (w_k, μ_k, π_k) , где w_k — единичная нормаль, μ_k — центр базового распределения признаков $N(\mu_k, I)$, π_k — текущая доля положительного класса. Метка задаётся правилом

$$g_k(x) = 1[w_k^T(x - \mu_k) > z], z = \Phi^{-1}(1 - \pi_0). \quad (2.6)$$

Объекты порождаются по схеме label shift: сначала $y \sim \text{Bernoulli}(\pi_k)$, затем x из $N(\mu_k, I)$, усечённого на полупространство класса y ; затем метка с вероятностью η инвертируется. Плотность $P_k(X)$ — это $N(\mu_k, I)$, перевзвешенная множителями π_k/π_0 и $(1 - \pi_k)/(1 - \pi_0)$ на полупространствах

классов. В исходном концепте все признаки одинаково информативны ($w_0=1/\sqrt{d}$), а $\mu_0=0$.

Каждый вид дрейфа меняет ровно одну группу параметров концепта (таблица 2.1).

Таблица 2.1 — Преобразования концепта для трёх видов дрейфа (метки без шума)

Вид	Преобразование	Меняется	Сохраняется
real	поворот w на угол ψ внутри подпространства затронутых признаков A	$P(y X)$	$P(y); P(X)$ при $\pi_k = \pi_0$
virtual	сдвиг $\mu \rightarrow \mu + \delta v$, где $v \perp w$ и носитель v лежит в A	$P(X)$	$P(y X), P(y)$
prior	замена доли класса $\pi_k \rightarrow \pi_{k+1}$	$P(y)$	$P(X y), P(y X)$

Две оговорки. Если до реального дрейфа был дрейф априорных вероятностей ($\pi_k \neq \pi_0$), то поворот нормали меняет и $P(X)$: перевзвешивание классов «переезжает» вместе с границей; валидатор отмечает такие события предупреждением W3. При шуме меток $\eta > 0$ дрейф априорных вероятностей меняет и наблюдаемое распределение $P(X|\tilde{y})$ для зашумлённой метки $\tilde{y}$.

Определение 3 (величина дрейфа). Величина обычного события e_k — расстояние полной вариации (total variation, TV) между совместными распределениями чистых данных до и после события:

$$m_k = TV(P_{k-1}, P_k) = \frac{1}{2} \sum_y \int |p_{k-1}(x, y) - p_k(x, y)| dx. \quad (2.7)$$

Утверждение 1 (единая шкала). Для каждого вида дрейфа величина (2.7) имеет точную форму. Для реального дрейфа

$$m = P_{x \sim P_{k-1}}(g_{k-1}(x) \neq g_k(x)), \quad (2.8)$$

то есть величина равна вероятностной мере той части входного пространства, где меняется метка; это адаптация понятия severity [6] к неограниченному пространству признаков. Для виртуального дрейфа $m = 2\Phi(\delta/2) - 1$, где δ — норма сдвига. Для дрейфа априорных вероятностей $m = |\pi_k - \pi_{k-1}|$.

Набросок доказательства. Для реального дрейфа при выборке из P_{k-1} отношение плотностей $p_k(x, y)/p_{k-1}(x, y)$ равно $1[g_k(x)=y]$: множители классов и нормировки совпадают, потому что базовые доли классов при повороте не меняются. Отсюда $TV = P_{k-1}(g_k \neq y) = P_{k-1}(g_{k-1} \neq g_k)$, в том числе при $\pi_k \neq \pi_0$. Для виртуального дрейфа плотность раскладывается в произведение компоненты вдоль w , от которой зависит класс, и независимой гауссовой компоненты в ортогональном дополнении; сдвиг затрагивает только вторую, поэтому TV совпадает с TV двух гауссиан, сдвинутых на δ . Для дрейфа априорных вероятностей классы занимают непересекающиеся полупространства, а $P(X|y)$ не меняется, поэтому TV равно $|\Delta\pi|$.

Утверждение 2 (калибровка реального дрейфа). При повороте нормали на угол θ

$$m(\theta) = (\pi_0 - \Phi_2(-z, -z; \cos \theta)) \left(\frac{\pi_k}{\pi_0} + \frac{1 - \pi_k}{1 - \pi_0} \right), \quad (2.9)$$

где $\Phi_2(\cdot, \cdot; \rho)$ — функция распределения стандартного двумерного нормального закона с корреляцией ρ . При $\pi_0 = \pi_k = 1/2$ отсюда следует $m = \theta/\pi$ — тот же геометрический факт, что и вероятность разделения двух векторов случайной гиперплоскостью [48].

Набросок доказательства. Величины $u = w_{k-1}^T(x - \mu)$ и $v = w_k^T(x - \mu)$ имеют стандартное двумерное нормальное распределение с корреляцией $\cos \theta$. Тогда $P(u > z, v \leq z) = \pi_0 - \Phi_2(-z, -z; \cos \theta)$, а перевзвешивание классов даёт множитель во вторых скобках. Функция Φ_2 вычисляется точно через T-функцию Оуэна [49].

Утверждение 3 (обратимость). Функция $m(\theta)$ строго возрастает на $[0, \pi]$, поскольку производная Φ_2 по корреляции положительна (формула Плакетта [50]). Следовательно, для любой достижимой величины m угол θ единствен и находится численно методом Брента.

Утверждение 4 (ограничение доли затронутых признаков). При повороте на угол ψ внутри подпространства A с сохранением нормы w_A

$$\cos \theta = 1 - q(1 - \cos \psi), q = \sum_{j \in A} w_j^2. \quad (2.10)$$

Отсюда наибольший угол $\theta_{\max} = \arccos(1 - 2q)$ и наибольшая достижимая величина $m_{\max} = m(\theta_{\max})$ (ограничение C10). Для исходного концепта $q = |A|/d \approx \alpha$. Например, при $\alpha = 0,3$ и балансе классов $m_{\max} = \arccos(0,4)/\pi \approx 0,369$.

Утверждение 5 (калибровка виртуального дрейфа). Сдвиг заданной величины m равен

$$\delta = 2\Phi^{-1}\left(\frac{1+m}{2}\right). \quad (2.11)$$

Расстояние Хеллингера H , которое используют Webb и соавторы, связано с TV неравенствами $H^2 \leq TV \leq H\sqrt{2-H^2}$. TV выбрано потому, что только эта метрика даёт одну и ту же интерпретируемую шкалу для всех трёх видов дрейфа.

Утверждение 6 (наблюдаемость реального дрейфа при шуме меток). Классификатор, оптимальный для концепта c_{k-1} , на зашумлённых метках концепта c_k ошибается с вероятностью $\eta + m(1 - 2\eta)$. Следовательно, прирост ошибки равен

$$m_{\text{eff}} = m(1 - 2\eta), \quad (2.12)$$

и TV для зашумлённых данных также равно $m(1 - 2\eta)$. При виртуальном дрейфе и дрейфе априорных вероятностей правило разметки не меняется, и ошибка старого классификатора остаётся равной η . Отсюда предупреждение W1 о почти ненаблюдаемом реальном дрейфе на фоне шума.

Для повторяющегося события величина не задаётся, а вычисляется: переход раскладывается по группам параметров на $\tilde{m} = (m_{\text{real}}, m_{\text{virtual}}, m_{\text{prior}})$, где m_{real} — доля смены метки, m_{virtual} — TV между базовыми распределениями признаков, $m_{\text{prior}} = |\Delta\pi|$.

Утверждения 1–6 проверены методом Монте-Карло независимо от кода фреймворка: каждая проверка строит выборку прямо из определения модели средствами NumPy и SciPy и сравнивает оценку с формулой с допуском в четыре стандартные ошибки. Все проверки выполнены. Калибровочная кривая (2.9) и граница достижимости $m_{\max}(\alpha)$ показаны на рисунке 2.1.

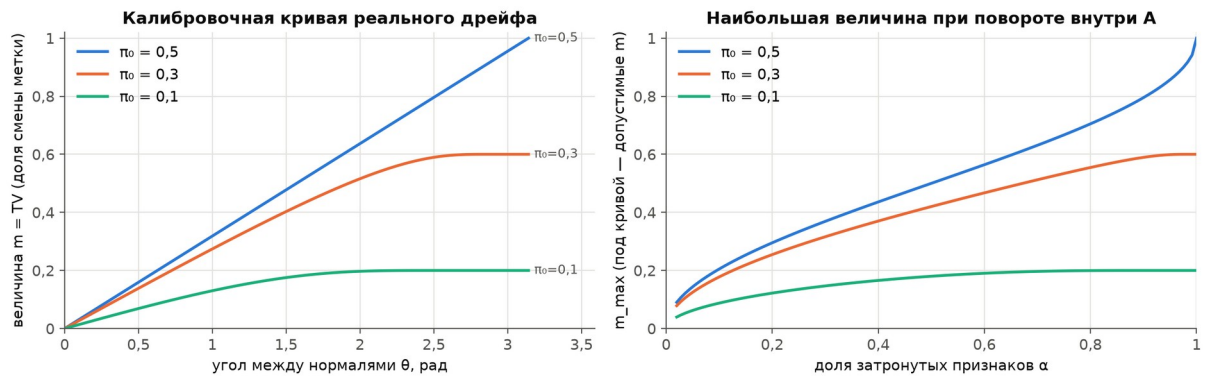


Рисунок 2.1 — Калибровочная кривая реального дрейфа и наибольшая достижимая величина при повороте внутри подпространства затронутых признаков

Из рисунка 2.1 следуют два вывода, существенных для формирования сценариев. Во-первых, при дисбалансе классов та же величина требует большего угла поворота, а наибольшая величина ограничена: при полном развороте $m_{\max} = 2\pi_0$ для $\pi = \pi_0$. Поэтому дисбаланс и величину нельзя выбирать независимо без проверки. Во-вторых, в принятой параметризации величина реального дрейфа ограничена сверху значением $m_{\max}(\alpha)$, растущим с долей затронутых признаков. Генератор без проверки молча выдал бы дрейф меньшей величины, чем заявлено; во фреймворке это нарушение ограничения C10. Другие преобразования весов дают иные ограничения: это свойство выбранной параметризации, а не дрейфа вообще.

2.4 Пространство параметров и таксономия сценариев

Параметризованное семейство сценариев задаётся отображением $\xi \mapsto S(\xi)$ точек пространства параметров $\mathcal{E} = \mathcal{E}_1 \times \dots \times \mathcal{E}_D$ в сценарии. Пространство по умолчанию для семейства `hyperplane_gauss` приведено в таблице 2.2; таблица строится из кода фреймворка при сборке записки, поэтому расхождение текста и реализации исключено. Варьируемых параметров 11; разогрев W и окно допуска Δ зафиксированы протоколом оценки.

Таблица 2.2 — Пространство параметров по умолчанию

Параметр	Обоз н.	Домен	По умолч.	Активен, если
длина потока	n	{10000, ..., 50000}	20000	всегда
размерность пространства признаков	d	{2, ..., 50}, лог.	10	всегда
исходная доля положительного класса $P(y=1)$	π	[0,05; 0,5]	0,5	всегда
вероятность инверсии метки	η	[0; 0,3]	0,0	всегда
число событий дрейфа	K	{1, ..., 5}	1	всегда
чередование концептов $A \rightarrow B \rightarrow A \rightarrow \dots$	alt	{False, True}	False	$n_drifts \geq 2$
форма перехода	φ	{sudden, gradual, incremental}	sudden	всегда
ширина перехода	ℓ	{100, ..., 4000}, лог.	1000	$form \in \{gradual, incremental\}$
вид дрейфа (какая компонента $P(X, y)$ меняется)	κ	{real, virtual, prior}	real	всегда
величина дрейфа	m	[0,02; 0,5]	0,2	всегда
доля признаков, затронутых дрейфом	α	[0,2; 1]	1,0	$kind \in \{real, virtual\}$
длина разогрева	W	{1000}	1000	всегда
окно допуска обнаружения	Δ	{2000}	2000	всегда

Условные параметры (например, ширина ℓ при внезапном дрейфе) в неактивном состоянии получают значение по умолчанию. Их координата в плане эксперимента при этом сохраняется, чтобы не нарушить стратификацию латинского гиперкуба и равномерность последовательности Соболя по остальным координатам.

Каждый сценарий относится к классам по восьми осям таксономии (таблица 2.3), по которым группируются результаты бенчмарка. Все величины измеряются в TV, поэтому ось «величина» сравнивает события разных видов на одной шкале. Пороги классов — договорённость авторов; они вынесены в константы модуля таксономии.

Таблица 2.3 — Оси таксономии сценариев

Ось	Классы
вид	real, virtual, prior и их сочетания
форма	sudden, gradual, incremental
повторяемость	есть возврат, нет возврата
величина	low ($m < 0,1$), medium ($0,1 \leq m < 0,3$), high ($m \geq 0,3$)
скорость	abrupt ($\ell = 0$), fast ($\ell < \Delta/2$), slow ($\ell \geq \Delta/2$)
охват	global ($\alpha = 1$ у всех событий real и virtual), partial, n/a
баланс	balanced ($\pi_0 \geq 0,4$), imbalanced
шум	clean ($\eta = 0$), noisy

2.5 Ограничения допустимости и истинная разметка

Сценарий допустим, если не нарушено ни одно из 14 ограничений C1–C14. Полный каталог ограничений и 4 предупреждений W1–W4 приведён в приложении А; он также строится из кода. Ограничения делятся на три уровня:

- ограничения события (C1, C3) — согласованность полей одного события, например «форма sudden тогда и только тогда, когда ширина равна нулю»; они проверяются при создании объекта и не зависят от семейства;

- ограничения сценария (C4–C7, C14) — согласованность событий между собой и с протоколом: дрейф не начинается во время разогрева, окна допуска разных событий не пересекаются, возврат ссылается на существующий концепт и действительно меняет распределение, протокол помещается в поток;

- ограничения семейства (C2, C8–C13) — возможности генератора: поддерживаемые виды и формы, достаточное число затронутых признаков, достижимость величины для каждого вида.

Пример зависимости от семейства — ограничение C2. Величина m обязательна для калиброванного семейства (hyperplane_gauss и внедрение дрейфа в реальные данные): генератор строит поток по заданной TV. Для некалиброванного семейства (классические генераторы river) величина запрещена: смена концепта там — смена встроенной функции, и её TV не

задаётся, а вычисляется. Если разрешить m для таких семейств, сценарий заявлял бы величину, которую генератор не реализует.

Ограничение С6 гарантирует, что окна допуска

$$\Lambda_k = [onset_k, end_k + \Delta) \quad (2.13)$$

разных событий не пересекаются, поэтому любое срабатывание детектора однозначно относится не более чем к одному событию. Ограничение С7 проверяет не только номер концепта, но и распределение: возврат к концепту, совпадающему с текущим по всем параметрам, отклоняется, поскольку такое событие нельзя обнаружить и оно исказило бы recall. Предупреждения W1–W4 сценарий не отклоняют, но учитываются при интерпретации результатов.

Исправление сценария (repair) выполняется так. Если нарушены ограничения С9–С12, валидатор предлагает ближайшее допустимое значение, например $m := m_{max}$. Если нарушены С4–С6, события переносятся по равномерному расписанию: свободное место делится поровну, поэтому раскладка допустима при любых ширинах переходов. Если следы событий не помещаются в поток, последние события отбрасываются. Исправление идёт последовательно от первого события к последнему, потому что каждое событие меняет состояние, от которого зависит допустимость следующих.

Истинная разметка (ground truth) сценария — набор записей

$$G = \{(onset_k, \tau_k, end_k, \phi_k, \kappa_k, m_k, \tilde{m}_k) : k = 1, \dots, K\}, \quad (2.14)$$

где $\tilde{m}_k = (m_{real}, m_{virtual}, m_{prior})$ — фактическая величина события по группам параметров. Для обычного события ненулевая только компонента его вида, и она равна заданной m_k ; для повторяющегося события компоненты вычисляются.

2.6 Протокол оценки и метрики

Протокол оценки — кортеж

$$\Omega = (h, a, W, \Delta, r), \quad (2.15)$$

где h — базовая онлайн-модель (по умолчанию гауссовский наивный байесовский классификатор из river [11]); $a \in \{\text{reset}, \text{none}\}$ — реакция на срабатывание детектора (сброс модели или только регистрация); W — разогрев; Δ — окно допуска; $r \in \{\text{onset}, \text{center}\}$ — точка отсчёта задержки.

Оценка ведётся по схеме prequential (test-then-train) [51]: модель сначала предсказывает $\hat{y}_t$, затем обучается на (x_t, y_t) , а детектор получает индикатор ошибки $e_t = 1[\hat{y}_t \neq y_t]$. Детектор получает ошибки начиная с $t=W$, чтобы не принимать кривую обучения модели за дрейф. После срабатывания детектор всегда начинается заново, независимо от того, как он обрабатывает срабатывание внутри себя; так все детекторы находятся в одинаковых условиях.

Пусть $\hat{D} = \{\hat{t}_1 < \hat{t}_2 < \dots\}$ — моменты срабатываний после разогрева. Правило сопоставления срабатываний с событиями таково. Событие k обнаружено (TP), если $\hat{D} \cap \Lambda_k \neq \emptyset$; момент обнаружения $\hat{t}_k^{\hat{D}} = \min(\hat{D} \cap \Lambda_k)$. Задержка при $r = \text{onset}$ — число объектов от начала перехода до срабатывания включительно:

$$\delta_k = \hat{t}_k^{\hat{D}} - \text{onset}_k + 1. \quad (2.16)$$

При $r = \text{center}$ задержка $\delta_k = \hat{t}_k^{\hat{D}} - \tau_k$ может быть отрицательной. Иначе событие пропущено (FN). Срабатывания вне всех окон считаются ложными тревогами FP_{out} , повторные срабатывания внутри окна — избыточными FP_{red} . Окна не пересекаются (C6), поэтому правило однозначно.

Метрики определены по CD-MOA [12] с обобщением на несколько событий (таблица 2.4). Здесь $T_{stable} = (n - W) - \sum_k |\Lambda_k|$ — число объектов вне окон допуска.

Таблица 2.4 — Метрики качества обнаружения

Метрика	Определение
Precision, Recall, F1	$P = T P / N_{det}$, $R = T P / K$, $F_1 = 2 P R / (P + R)$, где N_{det} — число срабатываний
MDR (доля пропусков)	$F N / K = 1 - R$
MTD (средняя задержка)	среднее δ_k по обнаруженным событиям
MTFA (время между ложными тревогами)	$T_{stable} / F P_{out}$
MTR	$(1 - MDR) \cdot MTF A / MTD$, только при $r = \text{onset}$
FAR (частота ложных тревог)	$1000 \cdot F P_{out} / T_{stable}$ — ложные тревоги на 1000 объектов
Дасс	разность prequential accuracy на $t \geq W$ с базовой линией без детектора

Для вырожденных случаев приняты соглашения: если срабатываний нет, precision не определена; при $TP=0$ полагается $F_1=0$, а MTD и MTR не определены; если ложных тревог нет, MTFA и MTR равны бесконечности, а при агрегировании MTFA ограничивается сверху значением T_{stable} .

Bifet показал, что оценка детектора по ассигасу классификатора, перезапускаемого при каждом срабатывании, вводит в заблуждение [4]. Поэтому основные метрики считаются относительно истинной разметки, ассигасу используется как дополнительная характеристика, а в число сравниваемых методов включаются базовые линии: NoDrift (без детектора), Periodic (сброс модели каждые 2000 объектов, без анализа данных) и Oracle (сброс в истинные моменты дрейфа). Детектор, срабатывающий слишком часто, может получить высокий recall за счёт окон допуска; это выявляют precision, MTFA и периодическая базовая линия.

Ещё одна особенность протокола: двусторонние детекторы (ADWIN, KSWIN, Page–Hinkley) срабатывают и на снижение ошибки, например после сброса модели. Такие срабатывания вне окон допуска считаются ложными тревогами, поэтому в экспериментах отдельно сравниваются двусторонний и односторонний варианты Page–Hinkley, а также политики $a = \text{reset}$ и $a = \text{none}$.

2.7 Метод формирования наборов сценариев

Постановка. Даны пространство параметров Ξ с D варьируемыми координатами, шаблон T — неполное описание сценария, задающее всё, что не варьируется, — и бюджет N . Требуется получить набор допустимых сценариев, равномерно покрывающий допустимую область

$$F = \{ \xi \in \Xi : S_T(\xi) \text{ удовлетворяет C1–C14} \}. \quad (2.17)$$

Набор строится конвейером: точки единичного куба $U \subset [0,1]^D$ отображаются по координатам в значения параметров $\xi = \Psi(u)$ с учётом условной активности, значения записываются в поля шаблона ($S = \Sigma_T(\xi)$), затем учитываются ограничения, и результат сохраняется как набор с манифестом.

Планы эксперимента. Сравниваются четыре плана при одном бюджете N :

- сетка — полный перебор уровней; число узлов растёт экспоненциально с числом осей;
- случайная выборка — независимые равномерные точки;
- латинский гиперкуб (LHS) [45] — каждая одномерная проекция содержит ровно одну точку в каждом из N слоёв;
- последовательность Соболя [46] — малое расхождение одновременно по всем осям; при $N=2^k$ план сбалансирован.

Сетка плохо масштабируется по размерности. Пусть категориальные параметры дают C сочетаний, а числовых осей q . Тогда при бюджете N на каждую числовую ось приходится

$$L = \lfloor (N/C)^{1/q} \rfloor \quad (2.18)$$

уровней. В пространстве по умолчанию $C=18$ и $q=8$, поэтому при $N < 18 \cdot 2^8 = 4608$ получается $L=1$: все числовые параметры остаются в середине доменов. Равномерность плана оценивается тремя показателями: центрированным L_2 -расхождением [52] (чем меньше, тем равномернее),

минимальным попарным расстоянием (maximin) и долей занятых слоёв одномерных проекций.

Учёт ограничений. Точка плана может оказаться вне F . Рассматриваются три способа.

1) Отклонение (reject): в набор входят только точки, для которых $\Psi(u) \in F$. Распределение принятых точек — исходное, условное на F , но набор меньше бюджета; доля принятых точек оценивает относительную меру F .

2) Проекция (gerair, подраздел 2.5): нарушенная координата заменяется ближайшим допустимым значением, события раскладываются заново. Размер набора равен N , но у распределения появляется атом на границе F : все точки с $m > m_{\max}$ переходят в $m = m_{\max}$.

3) Условные домены (adapt): для зависимой координаты домен сужается с учётом уже выбранных значений, и координата плана отображается в суженный домен:

$$D_j(\xi_{i,j}) = [l o_j, \min(h i_j, b_j(\xi_{i,j}))], \xi_j = D_j^{-1}(u_j), \quad (2.19)$$

где D_j^{-1} — отображение единичного отрезка в суженный домен (для ограничений снизу домен сужается аналогично: $[\max(l o_j, a_j(\xi_{i,j})), h i_j]$). Границы условных доменов приведены в таблице 2.5. Границы для α и m сообщает сам валидатор семейства как предложенное допустимое значение в нарушении, поэтому способ не зависит от семейства.

Таблица 2.5 — Границы условных доменов

Параметр	Граница	Откуда
число событий K	$K \leq \lfloor (n - W) / (\ell_{\min} + \Delta) \rfloor$	следы событий помещаются в поток (C4–C6)
ширина ℓ	$\ell \leq \lfloor (n - W) / K \rfloor - \Delta$	то же
доля признаков α	$\alpha \geq a(d)$	C9
величина m	$m \leq m_{\max}$	C10–C12

Утверждение 7. Пусть для каждой зависимой координаты множество допустимых значений при фиксированных остальных — отрезок,

содержащий lo_j (для ограничений сверху) или hi_j (для ограничений снизу). Тогда $\Psi_{adapt}(u) \in F$ при всех u , и у распределения непрерывных координат нет атомов на границе.

Доказательство. Координаты выбираются по порядку. Каждая лежит в суженном домене D_j , а он по условию целиком допустим при уже выбранных ξ_{ij} . Для непрерывной координаты отображение суженного домена непрерывно и строго монотонно по u_j ; прообраз любой точки — одна точка, поэтому множество положительной меры по u не переходит в одну точку, и атомов нет.

Условие утверждения выполнено для K и ℓ (ограничения линейны), для α (С9 задаёт порог снизу) и для одного события дрейфа — по монотонности $m(\theta)$ (утверждение 3), $TV(\delta)$ и $|\Delta\pi|$. При нескольких реальных событиях граница m_{max} следующего события зависит от величины предыдущих, и условие может нарушаться; тогда точка проверяется заново и при нарушении отклоняется. Цена способа: равномерность по u переходит в равномерность внутри условных доменов, а не по всей области F ; где граница b_j мала, плотность точек по ξ_j выше.

Способы различаются тем, чему остаётся верен набор. Отклонение сохраняет равномерность на F , но не план: точки из областей, где F «тоньше», выпадают, и баланс плана по исходным параметрам (например, по виду дрейфа) нарушается. Проекция и условные домены сохраняют все точки, а с ними — баланс плана по параметрам, которые они не меняют, но не сохраняют равномерность на F : проекция создаёт атом на границе, условные домены растягивают исправляемые координаты. Величина этих эффектов измерена в подразделе 4.3.

Шкала сложности. Один параметр $\lambda \in [0,1]$ управляет несколькими факторами. Каждый фактор интерполируется между значениями «легко» и «трудно» на своей шкале g_j (линейной или логарифмической):

$$\xi_j(\lambda) = g_j^{-1}((1-\lambda)g_j(\xi_j^{easy}) + \lambda g_j(\xi_j^{hard})). \quad (2.20)$$

По умолчанию с ростом λ величина m убывает от 0,4 до 0,05 (логарифмически), доля признаков α — от 1 до 0,2, ширина перехода растёт от 200 до 4000 (логарифмически), шум η — от 0 до 0,2, доля класса π_0 убывает от 0,5 до 0,1. Каждый фактор монотонен по λ по построению; профили каталога easy, medium и hard — точки $\lambda=0;0,5;1$. То, что рост λ действительно затрудняет обнаружение, — гипотеза; она проверяется в эксперименте Э4.

Наследование шаблонов. Шаблон с родителем раскрывается рекурсивно:

$$\text{resolve}(T) = \text{merge}(\text{resolve}(\text{parent}(T)), T). \quad (2.21)$$

Словари сливаются рекурсивно, остальные значения заменяются целиком, пустое значение (null) удаляет ключ родителя, способы задать события (явный список, компактная форма, блоки) взаимоисключающие; циклы наследования обнаруживаются.

Набор и версии. Идентификатор набора — хэш SHA-256 отсортированного списка идентификаторов сценариев:

$$ID_{suite} = \text{SHA256}(\text{sort}(id(S_1), \dots, id(S_N))). \quad (2.22)$$

Он не зависит от имени и версии набора. Сценарий i получает структурный seed и seed реализации, выведенные из пары (seed набора, i). Последовательность Соболя расширяема: при одном seed план из N точек — начальный отрезок плана из $2N$ точек. Поэтому набор Соболя можно увеличить, сохранив все уже прогнанные сценарии и их результаты; у латинского гиперкуба такого свойства нет.

2.8 Выводы по главе 2

1) Сценарий дрейфа формализован как кортеж $S = \langle B, E, \Omega, s \rangle$ с 14 ограничениями допустимости и 4 предупреждениями; структурная случайность отделена от случайности реализации.

2) Введена единая шкала величины дрейфа — расстояние полной вариации между совместными распределениями. Для калиброванного

семейства получены точные выражения величины для трёх видов дрейфа, калибровка реального дрейфа с учётом дисбаланса и доли затронутых признаков и ограничения достижимости величины.

3) Формализованы протокол оценки, правило сопоставления срабатываний с событиями и метрики обнаружения с соглашениями для вырожденных случаев.

4) Предложен метод формирования наборов сценариев по планам эксперимента с тремя способами учёта ограничений; для условных доменов доказано, что они дают допустимые точки без атомов на границе (утверждение 7). Метод дополнен шкалой сложности, наследованием шаблонов и идентификацией наборов по содержимому.

3 Проектирование и реализация фреймворка

3.1 Требования к фреймворку

Требования к фреймворку сформулированы по постановке задачи (подраздел 1.5). Функциональные требования приведены в таблице 3.1, нефункциональные — в таблице 3.2.

Таблица 3.1 — Функциональные требования

№	Требование	Где реализовано
Ф1	Описывать сценарий декларативно (YAML, JSON) и программно (Python API)	3.3
Ф2	Проверять допустимость сценария и исправлять недопустимые параметры	2.5, 3.3
Ф3	Порождать поток с контролируемым дрейфом: вид, форма, позиция, ширина, величина, число и расписание событий, шум, дисбаланс, размерность, доля затронутых признаков	3.4
Ф4	Внедрять дрейф в реальные наборы данных	3.4
Ф5	Строить истинную разметку автоматически: точки, интервалы и фактические величины дрейфа	3.4
Ф6	Формировать наборы сценариев автоматически: сетка, случайная выборка, латинский гиперкуб, последовательности Соболя, шаблоны, композиция, шкала сложности	2.7, 3.5
Ф7	Подключать детекторы через единый интерфейс	3.6
Ф8	Выполнять бенчмарк по схеме prequential с повторами и параллельно	3.6
Ф9	Вычислять метрики обнаружения и влияние на ассигасу, проводить статистическое сравнение	2.6, 3.6
Ф10	Формировать отчёты: таблицы, графики, экспорт в HTML, CSV, Markdown	3.6

Таблица 3.2 — Нефункциональные требования

№	Требование	Как обеспечивается
Н1	Воспроизводимость	детерминированный генератор по паре (сценарий, seed); идентификатор сценария по содержимому; зафиксированные версии библиотек
Н2	Проверяемость	семантика величины с аналитической калибровкой; модульные тесты; численная проверка утверждений
Н3	Расширяемость	реестры семейств генераторов и детекторов; интерфейсы на структурной типизации
Н4	Производительность	параллельные прогоны; кэш результатов по идентификатору; режимы FAST и FULL
Н5	Документированность	аннотации типов, строки документации, JSON Schema сценария, исполняемый ноутбук
Н6	Разделение авторства	у каждого модуля указан ответственный исполнитель

3.2 Архитектура

Фреймворк scendrift реализован как пакет языка Python 3.11 (модулей — 41, строк исходного кода — 7631). Он состоит из шести подпакетов (рисунок 3.1). Подпакеты scenario (формальная модель), formation (формирование наборов) и generators (генерация потоков) разработаны Будаевым К. В.; подпакеты detectors (детекторы), evaluation (протокол, раннер, метрики, статистика) и reporting (отчёты) — Гарифзяновым Т. Р. Внешние библиотеки показаны отдельно: они не являются вкладом авторов.

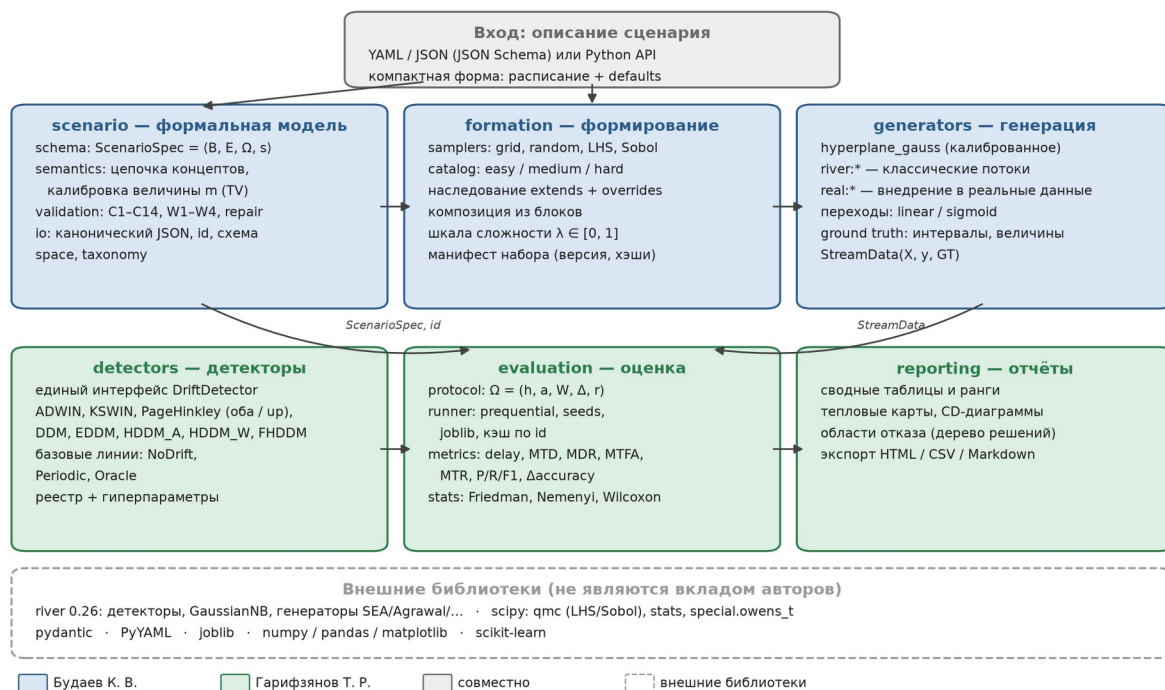


Рисунок 3.1 — Компоненты фреймворка scendrift и зоны ответственности исполнителей

Конвейер работы фреймворка показан на рисунке 3.2. Верхняя строка — формирование бенчмарка (пространство параметров, план эксперимента, сборка сценариев, проверка, набор с манифестом), нижняя — исполнение и анализ (генерация потоков, раннер, метрики, статистический анализ, отчёт).

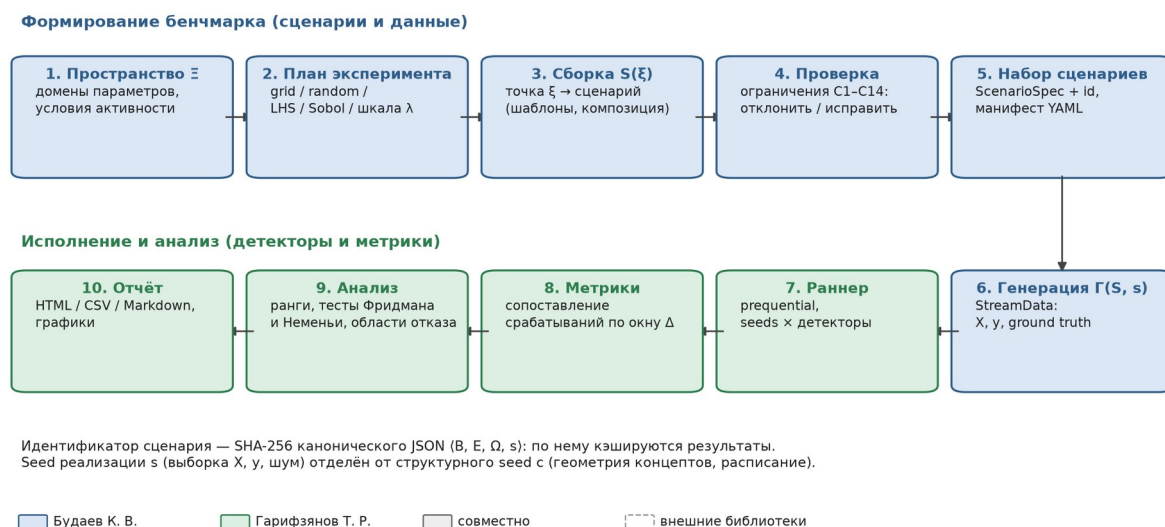


Рисунок 3.2 — Конвейер автоматического формирования и исполнения бенчмарка

Основные классы и интерфейсы показаны на рисунке 3.3. Сценарий ScenarioSpec объединяет конфигурацию потока StreamSpec, список событий DriftEvent и протокол оценки EvaluationSpec. Цепочка концептов ConceptChain вычисляется по сценарию без порождения данных и используется и валидатором, и генератором. Генератор возвращает поток вместе с истинной разметкой GroundTruth, которая вместе со срабатываниями детектора поступает на вход метрик.

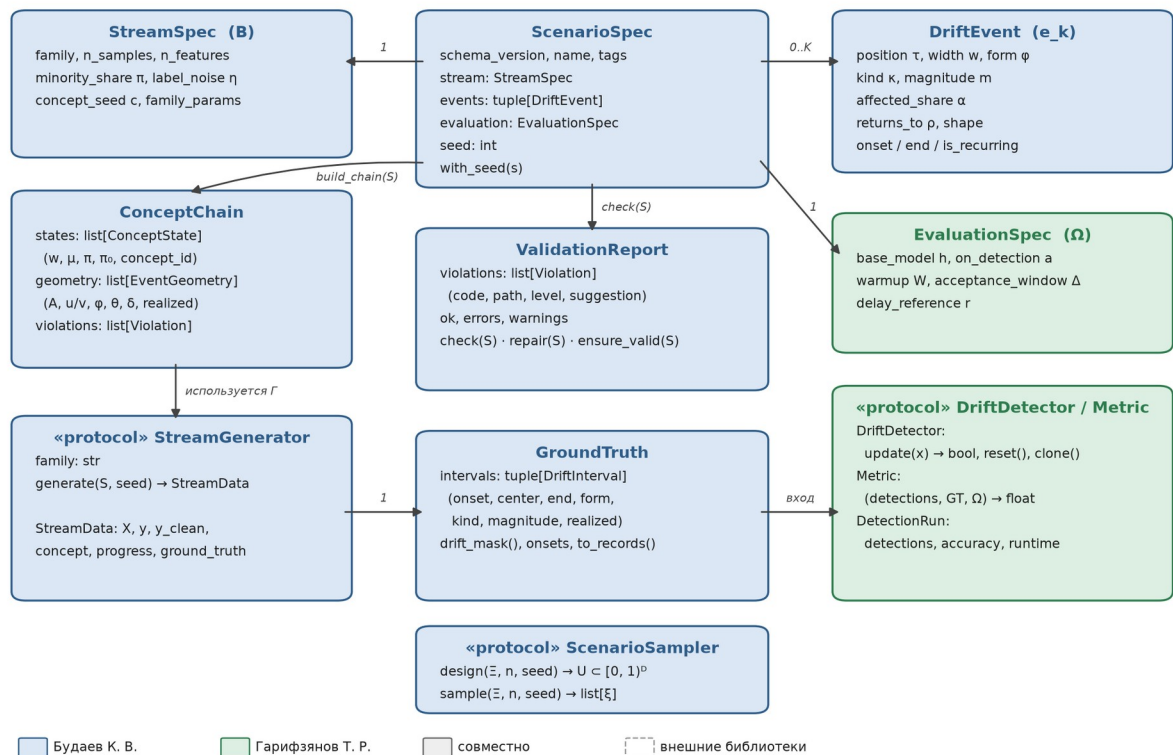


Рисунок 3.3 — Основные классы и интерфейсы фреймворка

Приняты следующие архитектурные решения:

1) Сценарий — неизменяемый объект-значение (модели pydantic с запретом изменения). Любая модификация (наследование, переопределение, исправление) создаёт новый сценарий с новым идентификатором, поэтому результаты не могут «перепутаться» со сценарием.

2) Проверка отделена от схемы. Схема проверяет домены полей и согласованность полей одного события (C1–C3). Ограничения, связывающие разные события и части сценария, и ограничения семейства (C4–C14)

вынесены в валидатор. Так модуль формирования может построить недопустимого кандидата и исправить его, а не отбрасывать точку плана.

3) Семантика вынесена из генератора. Цепочка концептов вычисляется отдельно, генератор только реализует выборку из рассчитанных концептов, поэтому валидатор и генератор используют один и тот же расчёт и не расходятся.

4) Расширение выполняется через реестры и интерфейсы. Новое семейство генераторов регистрируется вместе со своими ограничениями; новый детектор подключается адаптером, реализующим интерфейс DriftDetector (методы update, reset и clone), без наследования от классов фреймворка.

3.3 Декларативное описание и идентификация сценариев

Сценарий описывается в формате YAML или JSON в одной из двух форм. Каноническая форма — явный список событий; именно она хранится, хэшируется и подаётся генератору. Компактная форма — блок drifts с расписанием событий (равномерное, случайное или явное), параметрами по умолчанию и переопределениями отдельных событий; при загрузке она разворачивается в каноническую. Кроме того, описание может наследовать шаблон (ключ extends, в том числе профили каталога easy, medium и hard) или задавать события блоками. Пример компактного описания сценария с тремя событиями — постепенным реальным дрейфом, виртуальным дрейфом и возвратом к исходному концепту — приведён ниже; полный набор примеров дан в приложении Б.

```
stream: {n_samples: 20000, n_features: 10, minority_share: 0.3, label_noise: 0.05}
drifts:
  schedule: {mode: uniform, count: 3}
  defaults: {form: gradual, kind: real, width: 1000, magnitude: 0.2, affected_share: 0.5}
  overrides:
    - {index: 1, kind: virtual, magnitude: 0.4}
    - {index: 2, returns_to: 0}
seed: 42
```

Модели запрещают неизвестные поля, поэтому опечатка в файле приводит к ошибке, а не игнорируется. JSON Schema канонической формы генерируется из моделей автоматически и всегда соответствует коду; её можно подключить в редакторе для автодополнения и проверки файлов.

Идентификатор сценария — первые 12 шестнадцатеричных знаков хэша SHA-256 от канонического JSON смысловых полей $\langle B, E, \Omega, s \rangle$. Описательные поля (имя, теги, метаданные происхождения) в хэш не входят, поэтому переименование сценария не меняет идентификатор, а любое смысловое изменение — меняет. Дополнительно вычисляется идентификатор потока — хэш только тех полей, которые влияют на данные (без протокола оценки); он нужен для кэширования потоков. Сохранение и загрузка сценария выполняются без потерь, что проверяется тестами.

3.4 Генерация потоков

Общий конвейер генерации одинаков для всех семейств: проверка допустимости, временная разметка, порождение объектов, шум меток, построение истинной разметки и упаковка результата. Временная разметка задаёт для каждого объекта t тройку (src_t, dst_t, p_t) : переход идёт из состояния цепочки концептов src в состояние dst на доле пути p_t . Функция перехода бывает линейной или сигмоидной, как в МОА; сигмоида отнормирована так, чтобы весь переход лежал внутри интервала истинной разметки. Выборка объектов и шум используют разные подпотоки генератора псевдослучайных чисел, поэтому сценарии, отличающиеся только уровнем шума, порождают одни и те же признаки, и их результаты можно сравнивать попарно.

Калиброванное семейство `hyperplane_gauss` берёт состояния (w, μ, π) из цепочки концептов и порождает объекты по схеме label shift: сначала метку, затем проекцию на нормаль из усечённого нормального распределения (через обратную функцию распределения, без отбора), затем ортогональную составляющую. Инкрементальный переход интерполирует параметры для

каждого объекта: угол поворота, сдвиг и долю класса, а для возврата — нормаль сферической интерполяцией. Все вычисления векторизованы.

Классические генераторы river (SEA, Agrawal, STAGGER, Sine, Mixed) подключены как некалиброванные семейства. Классический сценарий — смена функции классификации генератора. Величину такой смены задать нельзя, но её можно измерить: при одном seed признаки у всех вариантов функции совпадают (это проверяется в коде), поэтому $TV = P(y_a \neq y_b)$ точно оценивается на общей эталонной выборке из 20 000 объектов. Фактическое значение попадает в истинную разметку.

Внедрение дрейфа в реальные данные реализовано для наборов Bananas (5300 строк, 2 признака) и Phishing (1250 строк, 9 признаков) из модуля river.datasets [11]. Концепт — дискретное распределение на строках набора (веса строк и их метки), поток порождается бутстрепом. Каждый вид дрейфа меняет ровно один множитель своего разложения $P(X, y)$ и точно сохраняет другой (таблица 3.3). Величина — TV совместных распределений на эмпирической мере с группировкой строк-дубликатов. Калибровка реального дрейфа выполняется точным перебором размера области, виртуального — бисекцией по параметру наклона, априорного — аналитически. На реальных данных события real и virtual могут попутно менять $P(y)$; если изменение больше 0,02, выдаётся предупреждение W4.

Таблица 3.3 — Виды дрейфа при внедрении в реальные данные

Вид	Преобразование	Сохраняется точно
real	инверсия меток в полупространстве $u^T z > c$	$P(X)$
virtual	экспоненциальный наклон весов строк, пропорциональный $e^{\lambda v^T z}$	$P(y X)$
prior	перевзвешивание классов	$P(X y)$

Модуль диагностики независимо оценивает по порождённым данным фактическую величину каждого события (долю смены метки для реального дрейфа, сдвиг проекций, пересчитанный в TV, для виртуального, разность долей класса для априорного) вместе со стандартной ошибкой, а также

эмпирическую кривую перехода. На нём основан эксперимент Э1 (подраздел 4.2).

3.5 Формирование наборов сценариев

Метод из подраздела 2.7 реализован в подпакете formation (таблица 3.4). Из внешних библиотек заимствованы только генераторы латинского гиперкуба и последовательностей Соболя и расчёт расхождения (модуль `scipy.stats.qmc` [14]).

Таблица 3.4 — Модули подпакета формирования наборов

Модуль	Назначение
samplers	планы эксперимента (сетка, случайная выборка, LHS, Соболев) и показатели равномерности
templates	шаблоны, наследование extends, встроенный каталог
mapping	отображение точки в сценарий; способы учёта ограничений reject, repair, adapt
compose	композиция сценария из блоков Stable, Drift, Recur
difficulty	шкала сложности λ и профили easy, medium, hard
suite	наборы, манифест, проверка целостности, сравнение версий

Все планы получают одно пространство и один бюджет N и возвращают точки единичного куба; сетка при бюджете N берёт наибольшее число уровней, при котором узлов не больше N . Модуль mapping записывает точку в поля шаблона, разворачивает компактную форму и применяет выбранный способ учёта ограничений; статус каждого сценария (исходно допустим, исправлен, адаптирован) сохраняется.

Композиция описывает сценарий как временную шкалу из блоков: Stable(L) — стабильный участок длины L , Drift(...) — событие дрейфа с параметрами, Recur($to = k$) — возврат к концепту k . Позиции событий и длина потока вычисляются автоматически. Пример композиции показан на рисунке 3.4: по вертикали отложен номер концепта, плавные участки соответствуют постепенным и инкрементальным переходам.

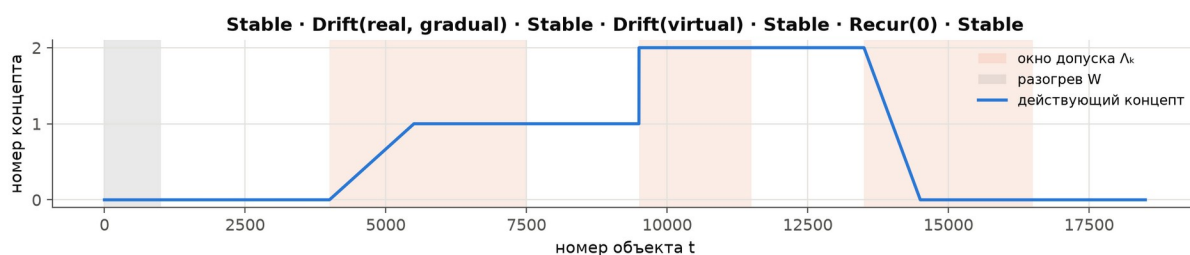


Рисунок 3.4 — Сценарий, собранный из блоков: стабильные участки, дрейф и возврат к исходному концепту

Набор сценариев формируется по декларативной конфигурации, в которой указаны шаблон, план, бюджет, seed и способ учёта ограничений (приложение Б). Манифест набора хранит версию схемы, версию пакета, идентификатор пространства параметров, конфигурацию, сводку формирования и для каждого сценария — идентификатор, файл и класс таксономии. При загрузке идентификатор каждого сценария пересчитывается по файлу, поэтому ручное изменение файла обнаруживается. Для основного бенчмарка сформирован набор `e2_fast` (идентификатор `ste-e61aa9a496c8`): план Соболя по 11 параметрам, размер набора — 64 сценариев по плану, из них исходно допустимых — 35, полученных условными доменами — 29; число покрытых классов таксономии — 62. Набор режима `FULL` (256 сценариев) включает набор режима `FAST` (64 сценария) как начальную часть, поскольку план Соболя расширяем.

3.6 Оценка детекторов

Адаптеры детекторов. Детекторы `river 0.26.1` приводятся к интерфейсу `DriftDetector`: метод `update` получает индикатор ошибки и возвращает признак срабатывания, методы `reset` и `clone` сбрасывают и копируют детектор. Реестр хранит детекторы с параметрами в имени, источником и направленностью реакции. Сравниваются девять детекторов (подраздел 1.3) и три базовые линии собственной реализации: `NoDrift`, `Periodic(2000)` и `Oracle`. Детекторы заимствованы; собственная часть — единый интерфейс, реестр и базовые линии.

Направленность реакции, указанная в реестре, проверена на синтетических потоках ошибок: первые 3000 объектов идут с частотой ошибок a , следующие 1000 — с частотой b ; считается доля потоков, в которых детектор сработал среди последних 1000 объектов. Для каждого случая есть контрольный поток без изменения. Результаты приведены в таблице 3.5.

Таблица 3.5 — Доля потоков со срабатыванием на последних 1000 объектах

Детектор	Реагирует на	Рост 0,1 → 0,35	Контроль 0,1	Падение 0,35 → 0,1	Контроль 0,35
ADWIN	оба	100 %	0 %	100 %	0 %
KSWIN	оба	40 %	0 %	30 %	50 %
PageHinkley	оба	100 %	0 %	100 %	5 %
PageHinkley-up	рост ошибки	100 %	0 %	0 %	5 %
DDM	рост ошибки	100 %	0 %	0 %	5 %
EDDM	рост ошибки	100 %	72,5 %	10 %	27,5 %
HDDM_A	рост ошибки	100 %	0 %	0 %	5 %
HDDM_W	рост ошибки	100 %	0 %	0 %	45 %
FHDDM	рост ошибки	100 %	0 %	0 %	20 %
FHDDM, вход e	—	7,5 %	0 %	97,5 %	27,5 %

Односторонние детекторы после падения ошибки срабатывают не чаще, чем на контрольном потоке, а ADWIN и двусторонний Page–Hinkley срабатывают после падения во всех потоках. Проверка выявила расхождение описания и реализации FHDDM в river 0.26.1: описание входа указывает индикатор ошибки, а реализация сигнализирует, когда среднее входа падает ниже своего максимума, — как в исходной статье, где в окне лежат индикаторы верного ответа. При подаче ошибок FHDDM срабатывает после падения ошибки в 97,5 % потоков, а после роста — лишь в 7,5 %. Поэтому в реестре FHDDM получает индикатор верного ответа $1 - e_i$; без этой поправки он обнаруживал бы улучшение модели, а не дрейф. Контрольные столбцы показывают фоновые ложные тревоги: у EDDM при частоте ошибок 0,1 они

возникают в 72,5 % потоков, у HDDM_W при частоте 0,35 — в 45 %. KSWIN с параметрами по умолчанию обнаружил рост ошибки лишь в 40 % потоков: он сравнивает 30 последних значений с 30 случайными из окна в 100 и требует статистики Колмогорова–Смирнова больше 0,1. Это свойство детектора, а не ошибка адаптера.

Базовая модель и раннер. Базовая модель повторяет вычисления гауссовского наивного байесовского классификатора river, но вычисляет ошибки векторно, блоками, по кумулятивным достаточным статистикам; цикл Python остаётся только у детектора. Ошибки векторной модели побитово совпадают с river на потоках трёх групп семейств со сбросами модели в произвольные моменты, а вычисления быстрее в десятки раз, поэтому время прогона определяют детекторы. Раннер выполняет протокол Ω (подраздел 2.6): модель работает по схеме test-then-train с первого объекта, детектор получает ошибки с $t=W$, после срабатывания детектор начинается заново, а при $a=\text{reset}$ сбрасывается и модель. Повторы выполняются по seed реализации, результаты кэшируются по идентификатору сценария, seed и имени детектора, а реализации сценариев обрабатываются параллельно (библиотека joblib).

Работа раннера на одном сценарии показана на рисунке 3.5: верхняя панель — скользящая частота ошибок модели без адаптации, нижняя — срабатывания всех детекторов и базовых линий относительно истинной разметки. Видно, что EDDM даёт серию ложных тревог сразу после разогрева, двусторонний Page–Hinkley — ложные тревоги после сбросов модели, а периодический сброс «обнаруживает» события только потому, что его срабатывания попадают в окна допуска.

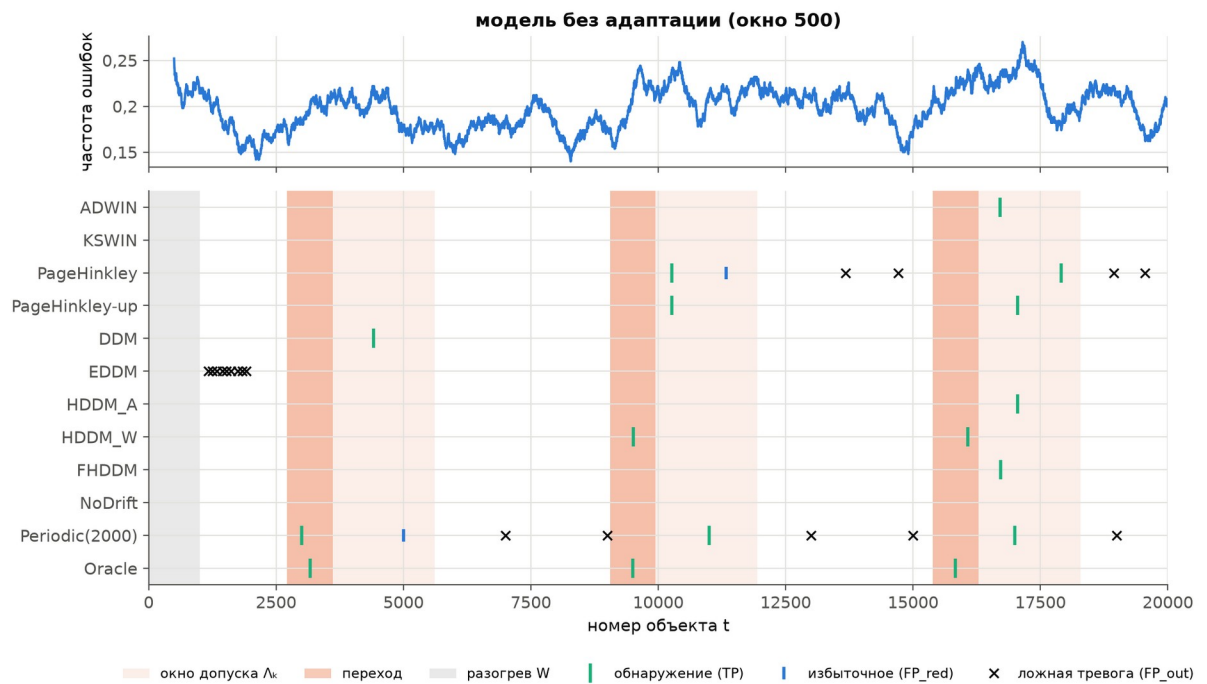


Рисунок 3.5 — Срабатывания детекторов на сценарии профиля medium (три постепенных реальных дрейфа, дисбаланс, шум 0,1)

Переходный процесс после сброса модели влияет на двусторонние детекторы: новая модель сначала ошибается часто, затем частота ошибок быстро падает, и двусторонний детектор может принять это падение за дрейф. На трёх профилях каталога при политике reset среднее число ложных тревог на прогон у ADWIN составляет 3,56 против 1,00 при политике none, у Page–Hinkley — 7,00 против 1,22; у односторонних детекторов разницы нет. Базовые линии ведут себя как задумано: Oracle получает $F1 = 1$, но повышает ассигасу только при реальном дрейфе (+0,057), а при виртуальном и априорном снижает её (−0,018 и −0,012), поскольку сброс выбрасывает модель, которая всё ещё верна. Поэтому разрыв до Oracle используется только для реального дрейфа.

Метрики и статистика. Модуль метрик реализует правило сопоставления и определения подраздела 2.6, включая соглашения для вырожденных случаев; значения сверены с ручным расчётом. Модуль результатов собирает таблицу «прогон — метрики и параметры сценария», в которой вычисляются разность ассигасу с NoDrift на той же реализации потока и разрыв до Oracle. Статистическое сравнение k методов на N

сценариях выполняется по Демшару [13]: средние ранги, критерий Фридмана [53] с поправкой Имана–Давенпорта, пост-хок Неменьи [54] с критической разностью

$$CD = q_{\alpha} \sqrt{\frac{k(k+1)}{6N}}, \quad (3.1)$$

попарный критерий Уилкоксона [55] с поправкой Холма [56]. Значение метрики в блоке усредняется по повторам, чтобы повторы не считались независимыми наборами данных. Квантили q_{α} вычисляются по распределению студентизированного размаха (функция `scipy.stats.studentized_range`) и совпадают с таблицей Демшара до 0,001.

Отчёты включают сводные таблицы «среднее $\pm$ стандартное отклонение» с десятичной запятой, CSV с сырыми результатами, сводку в Markdown, самодостаточный HTML-отчёт со встроенными таблицами и рисунками, CD-диаграммы и растры срабатываний. Пилотный прогон (сценариев — 16) проверил весь конвейер от набора сценариев до отчёта. KSWIN тратит в среднем 6,3 с на прогон, остальные детекторы — от 0,04 до 0,11 с.

3.7 Тестирование и воспроизводимость

Корректность фреймворка проверяется на трёх уровнях. Первый уровень — модульные тесты `pytest` (всего 261). Они проверяют функцию Φ_2 против многомерного нормального распределения SciPy, формулы величины против метода Монте-Карло, каждое ограничение C1–C14 на нарушающих его сценариях, исправление сценариев, сериализацию и «золотой» идентификатор, генераторы (функции перехода, независимость признаков от шума, фактическую величину в пределах четырёх стандартных ошибок, кривые перехода, точность внедрения в реальные данные), формирование наборов (стратификацию планов, способы учёта ограничений, наследование, композицию, шкалу сложности, целостность наборов), оценку (побитовое совпадение базовой модели с `river`, направленность детекторов, политики

раннера, кэш, метрики на ручном примере, квантили Неменьи) и анализ результатов.

Второй уровень — проверочные ячейки исполняемого ноутбука: численная проверка утверждений 1–6, точность калибровки, согласованность текста и кода, проверка утверждений выводов по каждому эксперименту. Выводы главы 4 сформулированы как проверяемые утверждения (всего 25); при перезапуске экспериментов (например, в режиме FULL) их выполнение проверяется автоматически, а сборка пояснительной записки останавливается, если хотя бы одно утверждение не выполнено. Отдельный тест сверяет ноутбук со свежей сборкой из исходных текстов.

Третий уровень — воспроизводимость: версии библиотек зафиксированы, глобальный seed задан, генерация детерминирована по паре (структурный seed, seed реализации), идентификаторы сценариев и наборов строятся по содержимому. Эксперименты выполняются в двух режимах: FAST (для ноутбука, минуты) и FULL (для сервера, полный объём). Числа в пояснительной записке подставляются из файлов результатов при сборке, поэтому после прогона FULL записка пересобирается без ручной правки.

3.8 Выводы по главе 3

1) Сформулированы функциональные и нефункциональные требования; спроектирована архитектура из шести подпакетов с разделением ответственности исполнителей и расширением через реестры и интерфейсы.

2) Реализован декларативный формат сценария с канонической и компактной формами, наследованием шаблонов, JSON Schema и идентификацией по содержимому.

3) Реализованы генерация потоков трёх групп семейств с истинной разметкой, формирование и версионирование наборов сценариев, оценка детекторов с векторной базовой моделью, метриками, статистикой и отчётами.

4) При проверке адаптеров обнаружено расхождение описания и реализации FHDDM в river 0.26.1 и введена поправка входа; измерено влияние сброса модели на ложные тревоги двусторонних детекторов.

5) Корректность обеспечена модульными тестами (всего 261), проверочными ячейками ноутбука и автоматической проверкой утверждений выводов.

4 Экспериментальное исследование

4.1 План экспериментов

Экспериментальное исследование отвечает на два вопроса: реализует ли фреймворк заявленные свойства сценариев (эксперимент Э1 и сравнение способов формирования наборов) и что показывает автоматически сформированный бенчмарк о детекторах дрейфа (эксперименты Э2–Э8). План экспериментов приведён в таблице 4.1.

Таблица 4.1 — План экспериментов

№	Эксперимент	Ответственный	Подраздел
Э1	Валидация генератора	Будаев К. В.	4.2
—	Сравнение планов и способов учёта ограничений	Будаев К. В.	4.3
Э2	Основной бенчмарк на автоматически сформированном наборе	Гарифзянов Т. Р.	4.4
Э3	Чувствительность и области отказа, «слепота» к виртуальному дрейфу	Гарифзянов Т. Р.	4.5
Э4	Шкала сложности	совместно	4.6
Э5	Статистическое сравнение	Гарифзянов Т. Р.	4.7
Э6	Сравнение с ручным набором классических потоков	Гарифзянов Т. Р.	4.8
Э7	Реальные данные с внедрённым дрейфом	Будаев К. В.	4.9
Э8	Влияние на ассигасу	Гарифзянов Т. Р.	4.10

Объём экспериментов задаёт режим запуска. В режиме FAST набор Э2 содержит 64 сценария с тремя повторами, сетка Э3 — 5×5 значений с тремя повторами, шкала Э4 — 11 значений λ с тремя структурными seed, Э6 и Э7 — по три повтора. В режиме FULL набор Э2 содержит 256 сценариев с пятью повторами, а остальные эксперименты — по десять повторов. Все числа главы получены запуском кода в режиме FAST и подставлены в текст из файлов результатов при сборке записки: в наборе Э2 сценариев — 64, повторов — 3, методов — 12 (девять детекторов и три базовые линии), всего прогонов — 2304. [ЗАПУСТИТЬ FULL НА СЕРВЕРЕ и пересобрать записку: числа обновятся автоматически, утверждения выводов будут проверены повторно.]

4.2 Э1. Валидация генератора

Вопрос эксперимента: совпадают ли параметры дрейфа в порождённых данных с заданными (величина, форма перехода, доля класса, шум) и воспроизводим ли поток по seed. Фактическая величина оценивается по данным, независимо от истинной разметки (подраздел 3.4), вместе со стандартной ошибкой SE. Генератор считается корректным, если нормированные отклонения $z = (\hat{m} - m)/SE$ распределены примерно как стандартный нормальный закон.

Заданная и фактическая величина на сетке. Рассматривается одно внезапное событие в середине потока ($d=10$, $\pi_0=0,3$, $\alpha=0,5$, без шума) для каждой пары «вид, величина» с несколькими seed реализации (рисунок 4.1). На сетке из 75 прогонов все $|z| \leq 3$ (максимум 2,36), стандартное отклонение z равно 0,99, средняя оценка отличается от заданной величины не более чем на 0,0025. Строки сетки с одним seed не независимы: при одном seed базовые случайные величины совпадают для всех m , поэтому независимую проверку даёт следующий опыт.

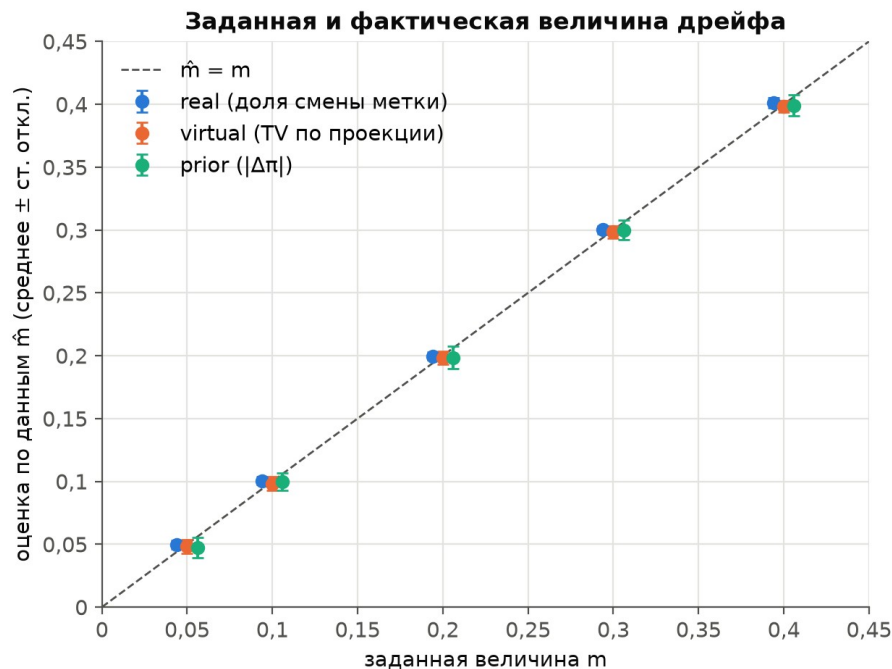


Рисунок 4.1 — Заданная и фактическая величина дрейфа на сетке (среднее и интервал по seed реализации)

Случайные сценарии. Берутся 900 случайных сценариев со случайными видом, величиной, долей класса $\pi_0 \in [0,05; 0,5]$, долей затронутых признаков и размерностью $d \in [3; 40]$. Объём выбран так, чтобы отличить стандартное отклонение 1 от заметно меньшего значения: выборочное стандартное отклонение по n значениям имеет относительную погрешность около $1/\sqrt{2n}$, то есть при наименьшем числе сценариев одного вида (286) — около 4 %. В пробных прогонах на 100 и 300 сценариях стандартное отклонение z для виртуального дрейфа составило 0,76 и 0,84, что ещё совместимо с 1 с учётом поправки на множественность, но уверенного вывода не давало; поэтому объём был увеличен. Для каждого вида строится доверительный интервал для стандартного отклонения по распределению χ^2 с поправкой Бонферрони на три вида (таблица 4.2).

Таблица 4.2 — Нормированные отклонения фактической величины от заданной на случайных сценариях

Вид дрейфа	Сценарие в	Среднее z	Ст. откл. z	ДИ для ст. откл.	Max $ \hat{m} - m $
реальный	291	-0,02	0,96	[0,87; 1,06]	0,011
виртуальный	286	0,07	0,95	[0,86; 1,06]	0,016
априорный	323	0,04	0,97	[0,88; 1,07]	0,022

Средние z лежат в пределах от -0,02 до 0,07, стандартные отклонения — от 0,95 до 0,97; доверительные интервалы накрывают 1 для всех трёх видов, а критерий Шапиро–Уилка не отвергает нормальность ($p = 0,23$, рисунок 4.2). Систематического смещения и ошибки в стандартной ошибке не обнаружено. Наибольшая абсолютная ошибка — у дрейфа априорных вероятностей (0,022), у которого разность долей класса имеет большую выборочную дисперсию; для реального дрейфа она равна 0,011, для виртуального — 0,016.

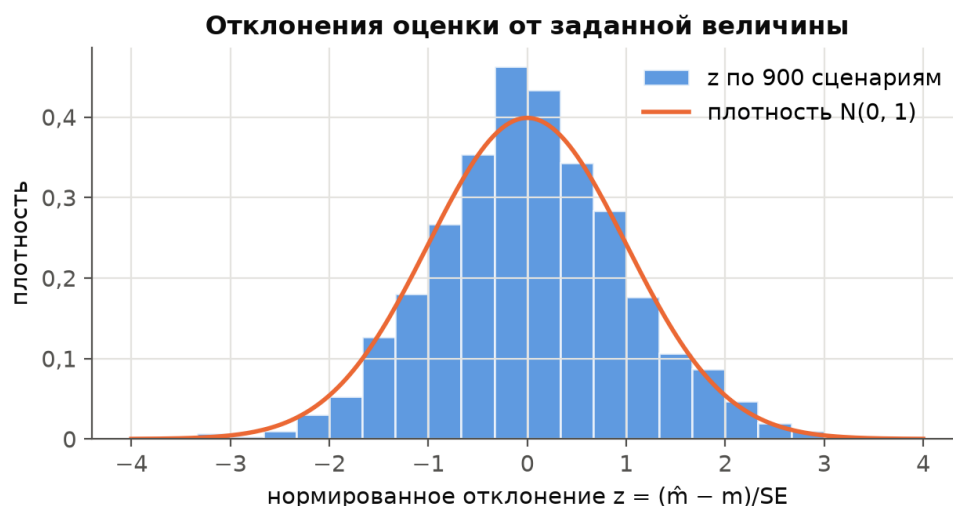


Рисунок 4.2 — Распределение нормированных отклонений на случайных сценариях и плотность стандартного нормального закона

Форма перехода. Для реального дрейфа величины 0,3 с шириной перехода 6000 сравниваются четыре варианта: внезапный, постепенный с линейной и с сигмоидной функцией, инкрементальный (рисунок 4.3). Эмпирическая кривая — скользящая (окно 800) доля объектов, у которых метка расходится с правилом старого концепта. Сравнивать её нужно с теоретической кривой, сглаженной тем же окном: иначе ступенька внезапного дрейфа даёт формальное расхождение 0,16, вызванное только сглаживанием. Для всех четырёх форм отклонение эмпирической кривой от сглаженной теоретической лежит в пределах от 0,026 до 0,040. При $m=0,3$ кривая инкрементального поворота почти линейна, поэтому по доле смены метки инкрементальный переход почти неотличим от постепенного линейного; формы различаются механизмом (при постепенном дрейфе объект порождён старым или новым концептом, при инкрементальном — промежуточным).

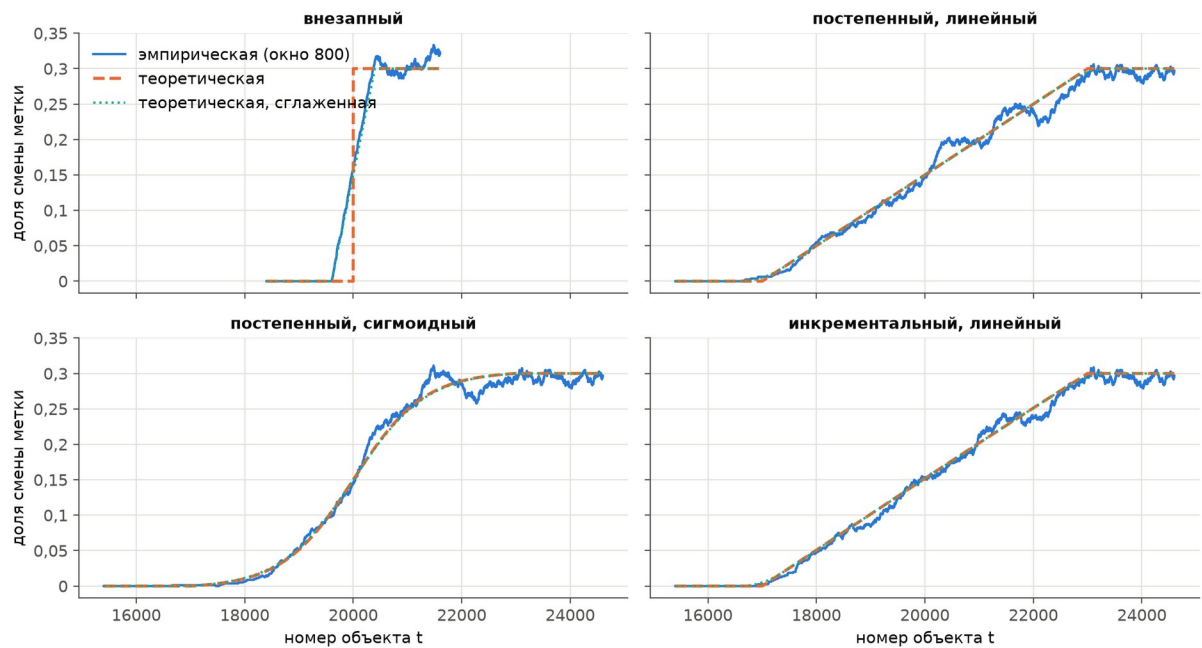


Рисунок 4.3 — Эмпирическая, теоретическая и сглаженная теоретическая кривые перехода реального дрейфа

Дисбаланс, шум и воспроизводимость. Доля класса воспроизводится с отклонением не более 1,4 SE. Доля инверсий меток равна 0,1037 при $\eta=0,1$ и 0,2052 при $\eta=0,2$; она одинакова при разных π_0 , потому что шум берётся из отдельного подпотока генератора псевдослучайных чисел. Для всех трёх групп семейств выполнены пять свойств воспроизводимости: одинаковый seed даёт побитово одинаковый поток, другой seed — другой поток, истинная разметка не зависит от seed реализации, сохранение сценария в YAML и загрузка дают тот же поток, изменение шума не меняет признаки.

Классические потоки на единой шкале. Для классических генераторов river вычислены TV смены соседних вариантов функции и побочное изменение доли класса (таблица 4.3).

Таблица 4.3 — Величина смены концепта классических генераторов river на единой шкале

Генератор	Смен концепта	TV смены	$ \Delta P(y) $
SEA	4	0,09 – 0,20	0,09 – 0,20
Agrawal	10	0,39 – 0,68	0,12 – 0,67
STAGGER	3	0,49 – 0,78	0,12 – 0,56
Sine	4	0,73 – 1,00	0,02 – 0,13
Mixed	1	1,00 – 1,00	0,08 – 0,08

«Смена функции» — внутренний параметр с неконтролируемой величиной от 0,09 до 1,0, несопоставимый между генераторами: из 22 смен лишь 4 имеют $TV < 0,25$. У SEA TV совпадает с $|\Delta P(y)|$: все изменённые метки меняются в одну сторону, поэтому каждая смена концепта SEA одновременно является сдвигом доли класса той же величины. Это количественно подтверждает первый пробел (подраздел 1.5): область малых и средних величин чистого реального дрейфа ручные бенчмарки почти не покрывают.

Реальные данные. Величина внедрённого дрейфа воспроизводится точно на эмпирической мере: для виртуального и априорного дрейфа — до округления, для реального — с точностью до массы одной строки (наибольшее отклонение 0,0004). Побочное изменение $P(y)$ у событий реального дрейфа достигает 0,040, предупреждение W4 выдано для 3 из 6 таких событий; у событий виртуального дрейфа побочное изменение не превышает 0,014. Порождение 10^5 объектов занимает доли секунды, поэтому генерация не является узким местом экспериментов.

4.3 Сравнение планов и способов учёта ограничений

Планы при равном бюджете. Сравняются планы из 128 точек в пространстве по умолчанию (11 варьируемых параметров), средние по 5 seed (таблица 4.4, рисунок 4.4).

Таблица 4.4 — Показатели равномерности планов эксперимента при бюджете 128

План	Точек	CD-расхождение	Maximin	Доля слоёв
сетка	18	1,1863	0,333	0,081
случайная выборка	128	0,0680	0,460	0,641
латинский гиперкуб	128	0,0404	0,498	1,000
Соболев	128	0,0213	0,510	1,000

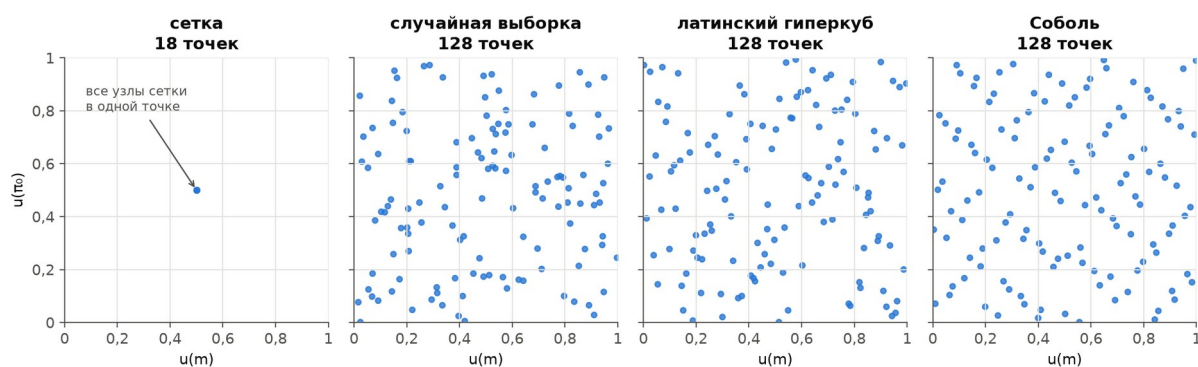


Рисунок 4.4 — Проекции планов из 128 точек на оси величины дрейфа и доли класса

Сетка вырождается в 18 узлов, перебирающих только категориальные параметры: все числовые параметры стоят в середине доменов, как и следует из формулы (2.18). У последовательности Соболя наименьшее центрированное расхождение (0,021 против 0,040 у латинского гиперкуба и 0,068 у случайной выборки) и наибольшее минимальное расстояние между точками; латинский гиперкуб и Соболев занимают все слои каждой оси. Поэтому основной план фреймворка — последовательность Соболя.

Способы учёта ограничений. Влияние способа учёта ограничений на распределение сценариев — свойство способа, а не плана, поэтому оно измерено методом Монте-Карло на 6000 случайных точках. Эталон — независимая случайная выборка того же размера, из которой оставлены только допустимые точки: она равномерна на допустимой области F . Для каждого способа измерены доля принятых точек, доля сценариев с величиной на границе достижимости (любое увеличение t нарушает C10–C12) и статистика Колмогорова–Смирнова по каждому параметру против эталона и

против исходного плана (таблица 4.5, рисунок 4.5). Порог значимости статистики при уровне 0,05 равен 0,025.

Таблица 4.5 — Свойства способов учёта ограничений (Монте-Карло; в скобках — параметр с наибольшей статистикой КС)

Способ	Принято	На границе	Доля real	КС против F	КС против плана
отклонение	79,4 %	0,0 %	23,8 %	0,022 (d)	0,084 (вид)
проекция	100,0 %	9,2 %	32,3 %	0,079 (вид)	0,072 (α)
условные домены	99,9 %	0,0 %	32,2 %	0,081 (α)	0,107 (α)

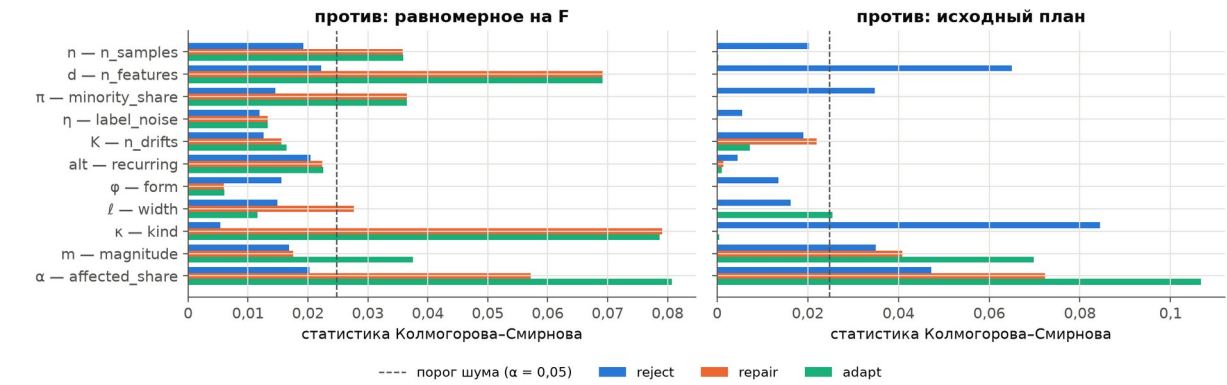


Рисунок 4.5 — Статистика Колмогорова–Смирнова по параметрам: против распределения, равномерного на допустимой области, и против исходного плана

Универсально лучшего способа нет. В пространстве по умолчанию недопустима примерно пятая часть точек: отклонение принимает 79,4 % точек. Отклонение сохраняет равномерность на допустимой области — отличие от эталона на уровне шума (КС 0,022), — но теряет часть бюджета и смещает баланс плана: чаще всего отклоняется реальный дрейф (ограничение C10), его доля падает с 32,3 % до 23,8 %, а доля дрейфа априорных вероятностей растёт с 34,2 % до 41,8 %. Проекция сохраняет весь бюджет, но 9,2 % сценариев оказываются на границе достижимости: их величина — максимально возможная при их геометрии, то есть набор смещается к «крайним» сценариям. Условные домены принимают 99,9 % точек, не создают атомов и сохраняют баланс плана по видам дрейфа; цена — наибольшее отклонение по доле признаков α (КС 0,107 против плана),

поскольку при малой размерности условный домен перемасштабирует α у всех точек.

Сочетания планов и способов при бюджете 128 сравниваются в таблице 4.6; покрытие оценивается числом различных классов таксономии по осям «вид, форма, величина, скорость, баланс, возврат».

Таблица 4.6 — Планы и способы учёта ограничений при бюджете 128 (среднее $\pm$ стандартное отклонение по 5 seed)

План	Способ	Принято	На границе	Классов таксономии
сетка	отклонение	100,0 %	0,0 %	18,0
сетка	проекция	100,0 %	0,0 %	18,0
сетка	условные домены	100,0 %	0,0 %	18,0
случайная выборка	отклонение	82,2 %	0,0 %	69,4 $\pm$ 4,4
случайная выборка	проекция	100,0 %	7,7 %	76,8 $\pm$ 4,8
случайная выборка	условные домены	100,0 %	0,0 %	77,0 $\pm$ 5,4
латинский гиперкуб	отклонение	78,3 %	0,0 %	68,0 $\pm$ 4,6
латинский гиперкуб	проекция	100,0 %	10,0 %	78,2 $\pm$ 2,8
латинский гиперкуб	условные домены	99,7 %	0,0 %	78,0 $\pm$ 2,8
Соболь	отклонение	77,2 %	0,0 %	69,0 $\pm$ 2,3
Соболь	проекция	100,0 %	9,7 %	81,4 $\pm$ 2,2
Соболь	условные домены	99,8 %	0,0 %	81,4 $\pm$ 1,8

Условные домены и проекция покрывают больше классов таксономии, чем отклонение (у Соболя 81,4 против 69,0); Соболь покрывает больше классов, чем латинский гиперкуб и случайная выборка, и с меньшим разбросом. Набор Соболя из 64 сценариев — начальная часть набора из 128, поэтому при увеличении бюджета все уже прогнанные сценарии и их результаты сохраняются; у латинского гиперкуба общих сценариев нет. Изменение файла сценария в сохранённом наборе обнаруживается при загрузке. Шкала сложности допустима на всей сетке λ при $d=20$; при $d=10$ на больших λ появляются недопустимые сценарии, поэтому в базовом шаблоне принято $d=20$. По умолчанию наборы формируются условными доменами, а статус каждого сценария сохраняется в манифесте, так что из набора можно выделить подмножество исходно допустимых точек.

4.4 Э2. Основной бенчмарк

Вопрос эксперимента: как детекторы *river* работают на наборе сценариев, равномерно покрывающем пространство параметров. Набор *e2_fast* сформирован автоматически (подраздел 3.5), всего событий дрейфа в нём — 191. Основной протокол — $a=\text{reset}$; дополнительно тот же набор прогоняется при $a=\text{none}$ (в режиме FAST без KSWIN ради времени). Блок статистического сравнения — сценарий: метрики усредняются по повторам. Средние значения метрик приведены в таблице 4.7, распределение F1 по сценариям — на рисунке 4.6.

Таблица 4.7 — Средние метрики на наборе Э2 (протокол *reset*; F1 — среднее $\pm$ стандартное отклонение по прогонам; MTD — в объектах, FAR — на 1000 объектов)

Метод	F1	Precision	Recall	MTD	FAR	Δacc
PageHinkley-up	$0,36 \pm 0,40$	0,59	0,35	987	0,02	0,011
FHDDM	$0,31 \pm 0,39$	0,59	0,33	979	0,03	0,010
HDDM_W	$0,28 \pm 0,33$	0,40	0,41	809	0,12	0,005
PageHinkley	$0,26 \pm 0,22$	0,26	0,53	1026	0,28	-0,009
HDDM_A	$0,24 \pm 0,37$	0,66	0,22	901	0,01	0,012
ADWIN	$0,22 \pm 0,32$	0,51	0,27	1013	0,06	0,006
DDM	$0,19 \pm 0,34$	0,49	0,19	879	0,02	0,012
KSWIN	$0,11 \pm 0,23$	0,37	0,11	1255	0,02	0,004
EDDM	$0,10 \pm 0,19$	0,08	0,18	770	0,29	0,007
NoDrift	$0,00 \pm 0,00$	—	0,00	—	0,00	0,000
Periodic(2000)	$0,38 \pm 0,20$	0,26	1,00	1077	0,46	-0,014
Oracle	$1,00 \pm 0,00$	1,00	1,00	324	0,00	0,006

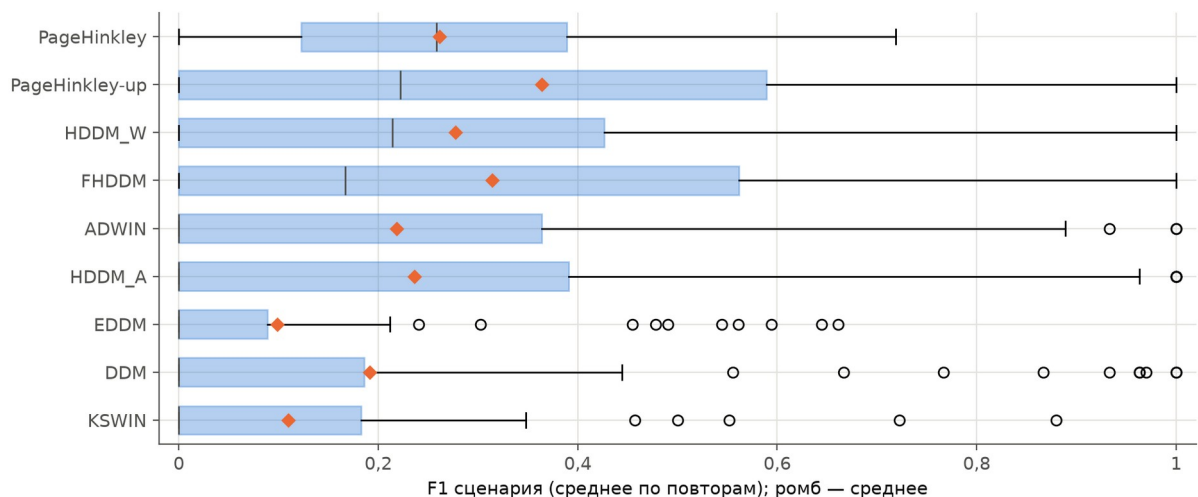


Рисунок 4.6 — Распределение F1 детекторов river по сценариям набора Э2 (F1 сценария — среднее по повторам; ромб — среднее по сценариям)

Качество обнаружения на равномерно покрывающем наборе низкое и неоднородное: средний F1 детекторов river — от 0,10 (EDDM) до 0,36 (PageHinkley-up); следом идут FHDDM (0,31) и HDDM_W (0,28). Разброс F1 по прогонам велик: стандартное отклонение сравнимо со средним, потому что в набор входят и лёгкие, и практически необнаружимые сценарии. Наименьшая частота ложных тревог — у HDDM_A.

Периодический сброс через 2000 объектов, не анализирующий данные, получает средний F1 0,38 — выше любого детектора — за счёт recall 1,00 при precision 0,26. Это количественное проявление «иллюзии прогресса» [4]: при окне допуска 2000 периодические срабатывания попадают в окна всех событий.

Политика адаптации (таблица 4.8). При сбросе модели у двустороннего Page–Hinkley среднее число ложных тревог на прогон — 6,2 против 1,6 без сброса: он реагирует на снижение ошибки после сброса. У EDDM соотношение обратное (4,7 против 20,2); механизм этого эффекта в работе не исследовался. При этом ранжирования детекторов по F1 при двух политиках согласованы: τ Кендалла [57] равно 0,86 ($p = 0,002$). Политика адаптации меняет абсолютные значения, но не порядок детекторов.

Таблица 4.8 — Влияние политики адаптации на F1 и среднее число ложных тревог на прогон

Детектор	F1 (reset)	F1 (none)	Ложных тревог (reset)	Ложных тревог (none)
PageHinkley-up	0,36	0,34	0,57	0,50
FHDDM	0,31	0,28	0,72	0,79
HDDM_W	0,28	0,22	2,59	3,08
PageHinkley	0,26	0,31	6,24	1,64
HDDM_A	0,24	0,18	0,20	0,21
ADWIN	0,22	0,21	1,18	0,45
DDM	0,19	0,13	0,35	0,49
EDDM	0,10	0,07	4,66	20,17

4.5 ЭЗ. Чувствительность и области отказа

Вопрос эксперимента: от каких параметров сценария зависит качество обнаружения и где детекторы отказывают. Эксперимент состоит из четырёх частей.

Сетка «величина × ширина». Базовый шаблон каталога ($d=20$, $\pi_0=0,5$, $\eta=0$, три постепенных реальных дрейфа); варьируются только $m \in \{0,03; 0,05; 0,1; 0,2; 0,4\}$ и $\ell \in \{100, 400, 1000, 2000, 4000\}$. Это контролируемый эксперимент: остальные параметры фиксированы (рисунок 4.7, таблица 4.9).

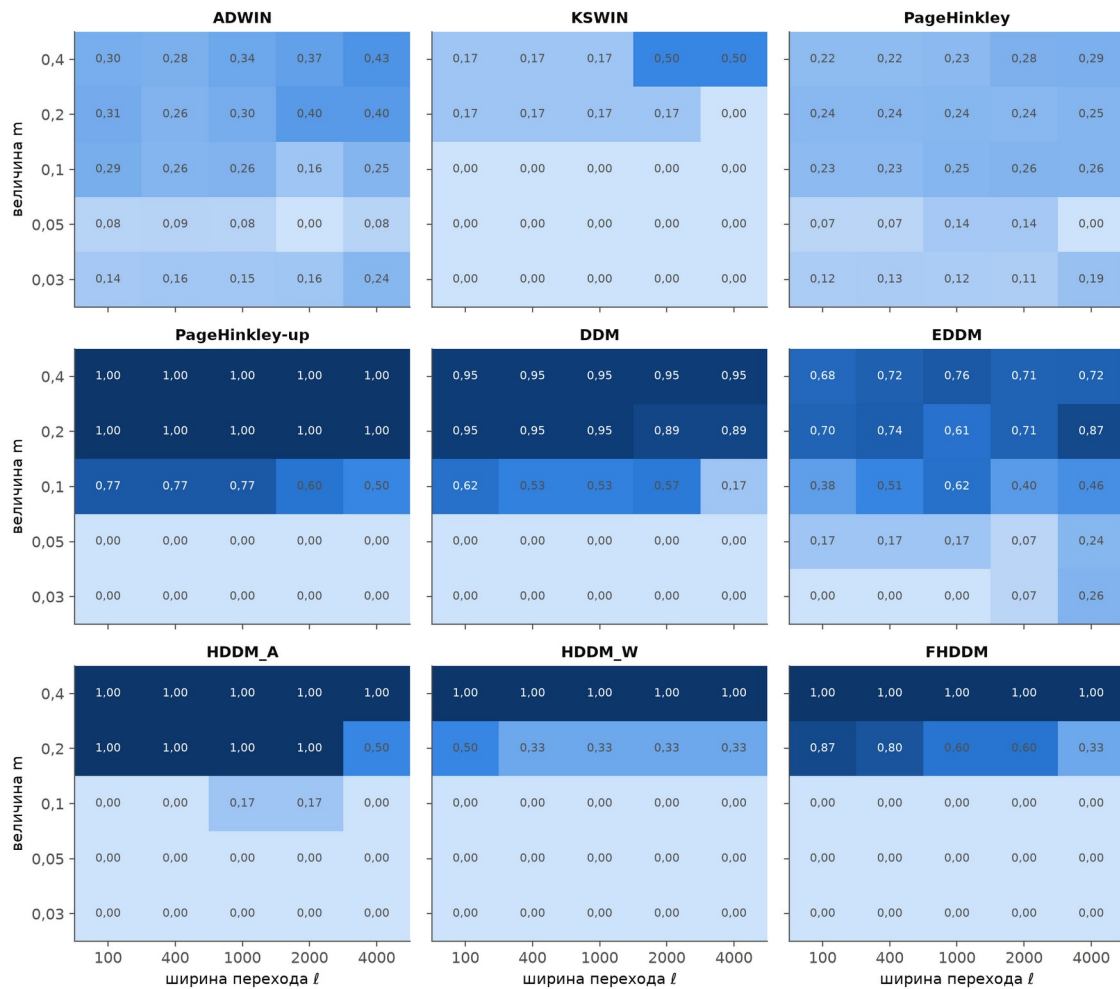


Рисунок 4.7 — Средний F1 на сетке «величина × ширина перехода» для каждого детектора

Таблица 4.9 — Связь F1 с величиной и шириной перехода на сетке (ρ Спирмена)

Детектор	$\rho(F1, m)$	$\rho(F1, \ell)$	F1 при $m = 0,03$	F1 при $m = 0,4$
PageHinkley-up	0,95	−0,03	0,00	1,00
FHDDM	0,89	−0,04	0,00	1,00
HDDM_W	0,89	−0,02	0,00	1,00
PageHinkley	0,70	0,34	0,13	0,25
HDDM_A	0,89	−0,01	0,00	1,00
ADWIN	0,82	0,12	0,17	0,34
DDM	0,95	−0,09	0,00	0,95
KSWIN	0,83	−0,04	0,00	0,30
EDDM	0,91	0,15	0,07	0,72

Величина — главный фактор: $\rho(F_1, m)$ лежит от 0,70 до 0,95, а связь с шириной перехода слабая ($|\rho| \leq 0,34$). Односторонние детекторы (PageHinkley-up, DDM, HDDM_A, HDDM_W, FHDDM) имеют $F1 = 0$ при

$m=0,03$ и не ниже $0,95$ при $m=0,4$: на единой шкале у них есть чётко выраженный порог обнаружения. У двусторонних детекторов (ADWIN, KSWIN, Page–Hinkley) и у EDDM F1 даже при $m=0,4$ заметно ниже (таблица 4.9).

Корреляции на наборе Э2. Для каждого детектора вычислен ρ Спирмена F1 с параметрами сценария (один сценарий — одно наблюдение, поправка Холма по параметрам; рисунок 4.8). На смешанном наборе связь F1 с величиной слабее, чем на сетке: ρ от $0,16$ до $0,36$, значимо после поправки — у FHDDM. Причина — в наборе доминирует вид дрейфа: виртуальный дрейф не обнаруживается при любой величине (см. ниже). Неожиданна значимая положительная связь F1 с шумом меток у FHDDM, HDDM_W, KSWIN: по утверждению 6 шум уменьшает наблюдаемый эффект реального дрейфа. Проверка механизма показала, что шум сильно связан с частотой ложных тревог этих детекторов (ρ не ниже $0,62$), но слабо — с превышением recall над случайным уровнем ($|\rho| \leq 0,22$). Шум увеличивает разброс потока ошибок, и тревоги, попавшие в окна допуска, засчитываются как обнаружения.

PageHinkley	+0,22	+0,11	+0,09	-0,03	+0,21	-0,18	+0,39*	-0,18
PageHinkley-up	+0,34	+0,02	-0,00	+0,01	+0,05	+0,08	+0,11	+0,16
HDDM_W	+0,20	+0,10	+0,08	-0,02	+0,45*	+0,08	+0,15	-0,02
FHDDM	+0,36*	+0,12	+0,12	+0,07	+0,44*	+0,05	+0,04	-0,08
ADWIN	+0,28	-0,02	+0,18	+0,12	-0,26	-0,11	+0,05	-0,07
HDDM_A	+0,27	-0,02	+0,12	+0,12	-0,01	+0,01	+0,09	-0,11
EDDM	+0,26	+0,01	+0,11	-0,02	-0,40*	-0,16	+0,35*	-0,26
DDM	+0,33	-0,05	+0,18	+0,21	-0,22	+0,00	+0,22	-0,01
KSWIN	+0,16	+0,24	-0,05	+0,06	+0,41*	+0,20	+0,27	-0,10
	m	ℓ	α	π_0	η	d	K	n

Рисунок 4.8 — Корреляции Спирмена F1 с параметрами сценария на наборе Э2 (звёздочка — значимо после поправки Холма)

«Слепота» к виртуальному дрейфу. Детектор по потоку ошибок может «обнаружить» событие случайно, ложной тревогой. Если ложные тревоги образуют пуассоновский поток с интенсивностью FAR, вероятность хотя бы одной тревоги в окне длины $\ell + \Delta$ равна

$$R_0 = 1 - \exp\left(-\frac{FAR(\ell + \Delta)}{1000}\right). \quad (4.1)$$

Наблюдаемый recall сравнивается с этим случайным уровнем отдельно для реального и виртуального дрейфа (таблица 4.10; вид — вид первого события сценария).

Таблица 4.10 — Recall R, случайный уровень R_0 по формуле (4.1) и превышение $R - R_0$ для реального и виртуального дрейфа

Детектор	R (real)	R_0 (real)	$R - R_0$ (real)	R (virtual)	R_0 (virtual)	$R - R_0$ (virtual)
PageHinkley-up	0,63	0,03	0,60	0,06	0,05	0,01
FHDDM	0,60	0,06	0,55	0,12	0,06	0,06
HDDM_W	0,64	0,18	0,46	0,22	0,21	0,01
PageHinkley	0,62	0,43	0,19	0,38	0,38	-0,01
HDDM_A	0,52	0,03	0,49	0,02	0,02	0,00
ADWIN	0,46	0,10	0,37	0,07	0,07	-0,01
DDM	0,46	0,06	0,39	0,00	0,04	-0,03
KSWIN	0,20	0,07	0,13	0,03	0,05	-0,02
EDDM	0,43	0,42	0,01	0,03	0,32	-0,29

На виртуальном дрейфе recall ни одного детектора не превышает случайный уровень больше чем на 0,06; на реальном превышение доходит до 0,6. Виртуальный дрейф по потоку ошибок на синтетических данных не обнаруживается: при неизменном $P(y|X)$ ошибка оптимальной модели не меняется (утверждение 6). У EDDM recall на реальном дрейфе превышает случайный уровень лишь на 0,005: его обнаружения объясняются частотой ложных тревог.

Правила отказа. Отказом считается средний по повторам recall меньше 0,5. Для каждого детектора по параметрам сценария обучено дерево решений глубины 2 (не меньше шести сценариев в листе) [58]; лист с преобладанием отказов выписывается как правило, а надёжность правил оценивается

пятикратной кросс-валидацией и сравнивается с ответом «класс большинства» (таблица 4.11).

Таблица 4.11 — Правила отказа детекторов на наборе Э2

Детектор	Доля отказов	Правило (лист с наибольшим числом отказов)	Точность (CV)	Класс большинства
PageHinkley-up	0,67	вид = virtual	0,70	0,67
FHDDM	0,62	$\eta \leq 0,158$ и вид $\neq$ real	0,65	0,62
HDDM_W	0,56	$\eta \leq 0,158$ и вид $\neq$ real	0,69	0,56
PageHinkley	0,41	$m \leq 0,12$ и $\pi_0 \leq 0,274$	0,58	0,59
HDDM_A	0,78	вид $\neq$ real и $d \leq 15,5$	0,92	0,78
ADWIN	0,75	вид $\neq$ real и $\eta > 0,0519$	0,81	0,75
DDM	0,84	вид $\neq$ real и $\ell \leq 1850$	0,95	0,84
KSWIN	0,95	$m \leq 0,351$	—	0,95
EDDM	0,84	вид $\neq$ real и $\pi_0 > 0,117$	0,89	0,84

Правила в основном выделяют вид дрейфа («вид $\neq$ real»): так устроены правила у 7 детекторов из девяти. Точность правил превосходит ответ «класс большинства» на 0,05 и более лишь у 4 детекторов; устойчивые правила по числовым параметрам при объёме FAST не выделяются. Методика описания областей отказа работает, но для надёжных многопараметрических правил нужен объём FULL.

4.6 Э4. Шкала сложности

Вопрос эксперимента: отражает ли шкала сложности λ (подраздел 2.7) реальную трудность обнаружения и какие факторы её определяют. Для 11 значений λ от 0 до 1 с несколькими структурными seed прогоняются все детекторы; шкала отражает сложность, если F1 убывает с ростом λ (рисунок 4.9, таблица 4.12). Дополнительно приводится доля шагов, на которых средняя кривая $F_1(\lambda)$ не возрастает.

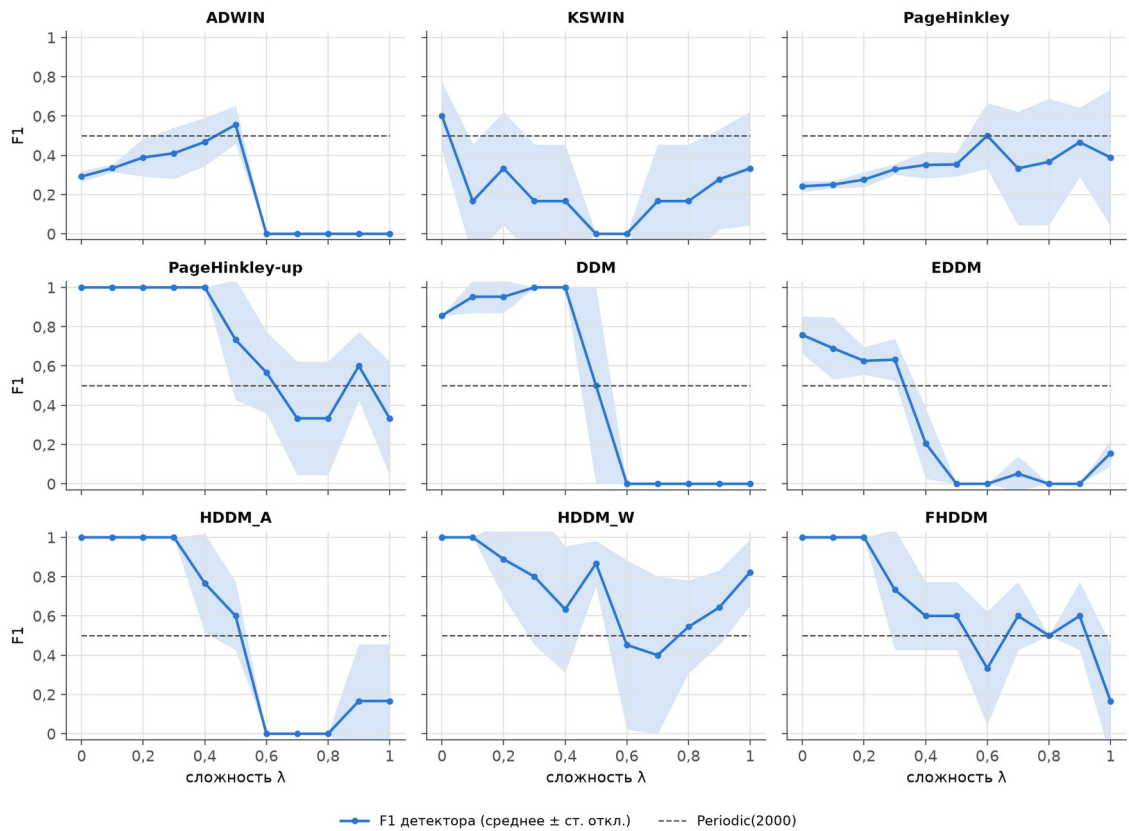


Рисунок 4.9 — Средний F1 детекторов вдоль шкалы сложности

Таблица 4.12 — Связь F1 с параметром сложности λ

Детектор	$\rho(F1, \lambda)$	p (Холм)	Невозрастающих шагов	F1 при $\lambda = 0$	F1 при $\lambda = 1$
PageHinkley-up	-0,81	< 0,001	0,9	1,00	0,33
FHDDM	-0,75	< 0,001	0,8	1,00	0,17
HDDM_W	-0,52	0,006	0,6	1,00	0,82
PageHinkley	0,52	0,006	0,2	0,24	0,39
HDDM_A	-0,86	< 0,001	0,9	1,00	0,17
ADWIN	-0,67	< 0,001	0,5	0,29	0,00
DDM	-0,77	< 0,001	0,8	0,86	0,00
KSWIN	-0,16	0,37	0,6	0,60	0,33
EDDM	-0,74	< 0,001	0,7	0,76	0,16

F1 значительно убывает с ростом λ у 7 из девяти детекторов (незначимо — у KSWIN), однако кривые немонотонны: доля невозрастающих шагов у них — от 0,5 до 0,9. У двустороннего Page–Hinkley F1 растёт с λ ($\rho = 0,52$): на лёгких сценариях с резкими сильными изменениями он чаще ложно срабатывает на переходный процесс после сброса модели.

Немонотонность связана с окнами допуска. Ширина перехода растёт с λ , а вместе с ней — окна допуска длины $\ell + \Delta$ (таблица 4.13). При $\lambda=0$ окна

покрывают 34,7 % потока после разогрева, при $\lambda=1$ ($\ell = 4000$) — 94,7 %, и F1 засчитывает случайные тревоги: например, у HDDM_W F1 при $\lambda=1$ равен 0,82.

Таблица 4.13 — Окна допуска вдоль шкалы сложности

λ	Ширина ℓ	Доля потока в окнах допуска	Recall (river)	Случайный recall
0,0	200	34,7 %	0,94	0,23
0,3	491	39,3 %	0,80	0,20
0,5	894	45,7 %	0,44	0,19
0,7	1628	57,3 %	0,17	0,14
0,9	2965	78,4 %	0,26	0,25
1,0	4000	94,7 %	0,25	0,11

Анализ по одному фактору. Каждый фактор шкалы проходит свой диапазон от «легко» до «трудно», а остальные стоят в центральной точке шкалы ($\lambda=0,5$); центральная точка выбрана потому, что вокруг крайней точки «легко» многие сочетания недостижимы (например, при $m=0,4$ и $\pi_0=0,1$ реальный дрейф не может превысить $TV = 0,2$). Результаты сведены в таблице 4.14.

Таблица 4.14 — Анализ по одному фактору: $\rho(F1, \lambda_j)$ по детекторам river (без KSWIN в режиме FAST)

Фактор	Средний $\rho(F1, \lambda_j)$	Значимо отрицательных	Детекторов
величина m	-0,82	8	8
доля признаков α	0,03	0	8
ширина ℓ	-0,07	0	8
шум η	0,04	1	8
доля класса π_0	-0,18	1	8

Величина дрейфа значимо затрудняет обнаружение у всех детекторов (8 из 8). Доля затронутых признаков при фиксированной TV не даёт ни одной значимой связи — это согласуется с идеей единой шкалы: величина в TV уже учитывает, сколько признаков затронуто. Для ширины, шума и дисбаланса согласованного эффекта при объёме FAST не обнаружено. Отсюда предложение по развитию шкалы: оставить в ней прежде всего величину и ограничить рост ширины перехода, чтобы окна допуска не покрывали поток.

4.7 Э5. Статистическое сравнение

Вопрос эксперимента: какие различия между детекторами на наборе Э2 статистически значимы. Сравниваются девять детекторов *river* по четырём метрикам: F1, MTD, FAR и Δ accuracy (для MTD исключаются блоки, где какой-либо детектор ни разу не обнаружил событие). Результаты критерия Фридмана с поправкой Имана–Давенпорта приведены в таблице 4.15, CD-диаграммы Неменьи — на рисунке 4.10.

Таблица 4.15 — Критерий Фридмана и критическая разность Неменьи ($\alpha = 0,05$)

Метрика	Блок в	$\chi^2 F$	p	FF	CD	Лучший по рангу	Худший по рангу
F1	64	93,6	< 0,00 1	14,1	1,50	PageHinkley-up	KSWIN
MTD	9	32,0	< 0,00 1	6,4	4,00	FHDDM	KSWIN
FAR	64	250,0	< 0,00 1	60,1	1,50	HDDM_A	EDDM
Δ accuracy	64	112,6	< 0,00 1	17,8	1,50	DDM	PageHinkley

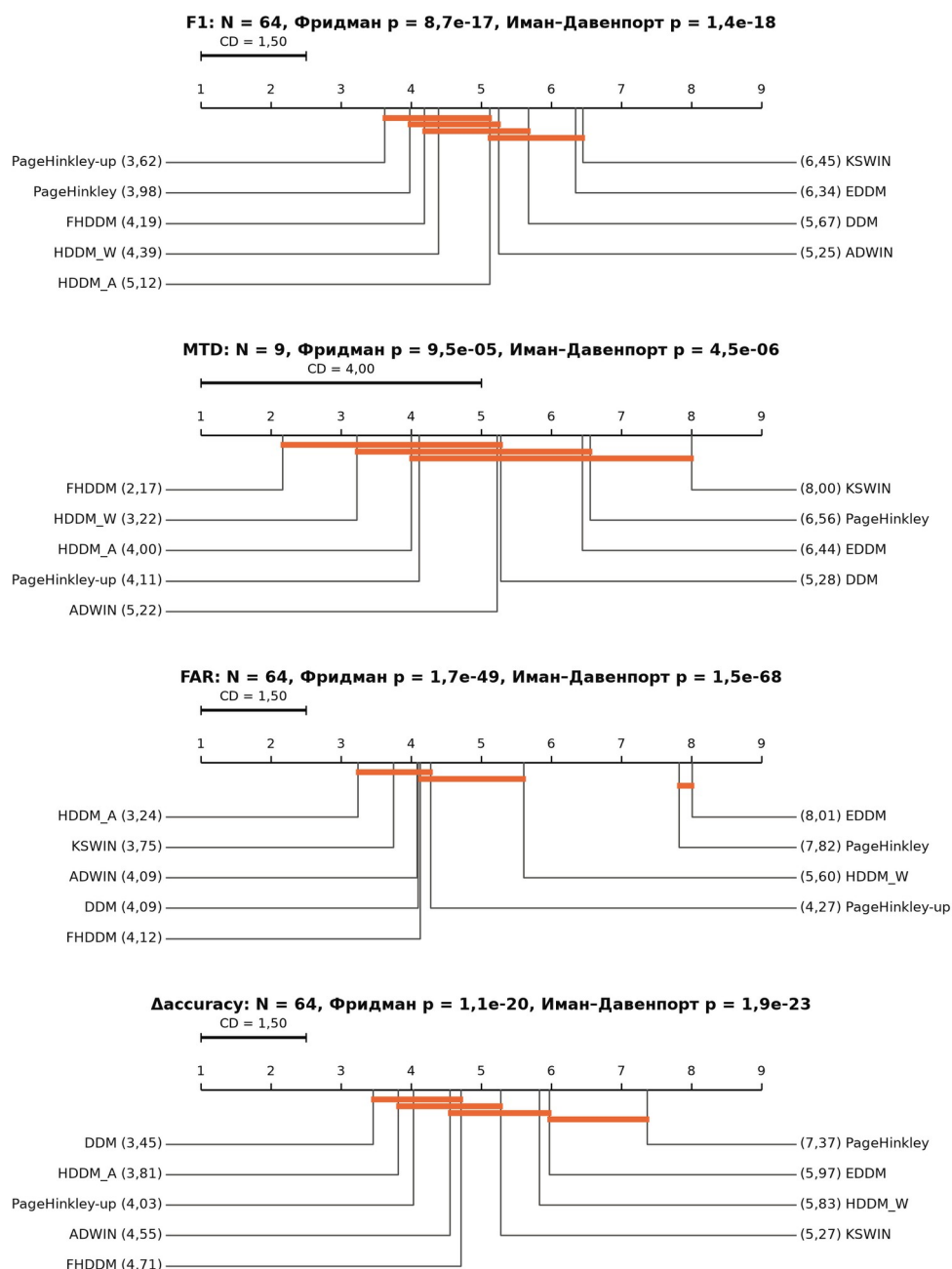


Рисунок 4.10 — CD-диаграммы Неменьи для F1, MTD, FAR и Дассигасу на наборе Э2

Критерий Фридмана отвергает равенство детекторов по всем четырём метрикам ($p < 0,001$). Для MTD блоков, где все детекторы обнаружили хотя бы одно событие, всего 9, поэтому критическая разность велика и выводы по задержке ненадёжны. По F1 попарный критерий Уилкоксона с поправкой Холма обнаруживает различие в 15 парах из 36.

Сравнение с периодическим сбросом (таблица 4.16) количественно подтверждает «иллюзию прогресса». По F1 7 из девяти детекторов значимо

хуже периодического сброса, не обнаружено различия у PageHinkley-up, FHDDM, и ни один детектор не лучше. По Δ accuracy, наоборот, значимо лучше периодического сброса 9 детекторов из девяти. Следовательно, F1 с окнами допуска нельзя использовать как единственную метрику: его нужно дополнять периодической базовой линией, частотой ложных тревог и сравнением со случайным уровнем.

Таблица 4.16 — Сравнение детекторов с периодическим сбросом (критерий Уилкоксона, поправка Холма по девяти детекторам)

Детектор	F1: медиана разности	p (Холм)	Вывод	Δ асс: медиана разности	p (Холм)	Вывод
PageHinkley-up	−0,16	0,69	не обнаруже но	0,020	< 0,001	лучше
FHDDM	−0,14	0,25	не обнаруже но	0,020	< 0,001	лучше
HDDM_W	−0,11	0,02	хуже	0,014	< 0,001	лучше
PageHinkley	−0,11	< 0,001	хуже	0,006	< 0,001	лучше
HDDM_A	−0,25	0,02	хуже	0,022	< 0,001	лучше
ADWIN	−0,22	< 0,001	хуже	0,017	< 0,001	лучше
DDM	−0,25	< 0,001	хуже	0,023	< 0,001	лучше
KSWIN	−0,26	< 0,001	хуже	0,018	< 0,001	лучше
EDDM	−0,28	< 0,001	хуже	0,019	< 0,001	лучше

4.8 Э6. Сравнение с ручным набором классических потоков

Вопрос эксперимента: совпадают ли выводы о детекторах на ручном наборе классических потоков и на автоматически сформированном наборе. Ручной набор — 10 потоков: пять классических генераторов river (SEA, Agrawal, STAGGER, Sine и Mixed) с двумя формами перехода (внезапной и постепенной шириной 1000, как у ConceptDriftStream в MOA), по три события, $n=20\,000$, параметры river по умолчанию. План работы предусматривал также генераторы Hyperplane и RandomRBFDrift, но в них дрейф непрерывный, без дискретных событий, поэтому для них не определены истинная разметка и метрики подраздела 2.6, и в набор они не включены. Результаты приведены в таблицах 4.17 и 4.18 и на рисунке 4.11.

Таблица 4.17 — F1 и средние ранги на ручном наборе и на наборе Э2

Метод	F1 (ручной набор)	F1 (набор Э2)	Ранг (ручной)	Ранг (Э2)
PageHinkley-up	0,98	0,36	2,30	3,62
FHDDM	0,75	0,31	4,50	4,19
HDDM_W	0,62	0,28	5,55	4,39
PageHinkley	0,72	0,26	5,45	3,98
HDDM_A	0,93	0,24	3,10	5,12
ADWIN	0,83	0,22	4,60	5,25
DDM	0,88	0,19	4,40	5,67
KSWIN	0,44	0,11	6,90	6,45
EDDM	0,35	0,10	8,20	6,34
NoDrift	0,00	0,00	—	—
Periodic(2000)	0,50	0,38	—	—
Oracle	1,00	1,00	—	—

Таблица 4.18 — Покрывание пространства сценариев ручным набором и набором Э2

Набор	Событий	TV < 0,1	TV < 0,25	Медиана TV	Классов	Видов
ручной набор	30	6,7 %	20 %	0,59	2	1
набор Э2	191	20,9 %	53,4 %	0,24	62	3

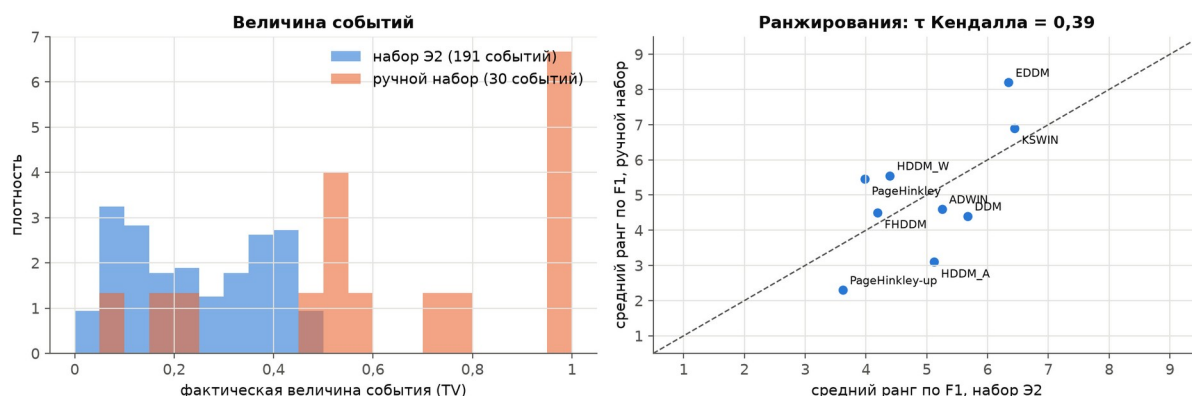


Рисунок 4.11 — Распределение величины событий и сравнение рангов детекторов на ручном наборе и наборе Э2

Ручной набор завышает качество: средний F1 детекторов на нём — от 0,35 до 0,98, на наборе Э2 — от 0,10 до 0,36. Согласие ранжирований не подтверждено: $\tau = 0,39$ ($p = 0,18$). Ручной набор покрывает классов таксономии — 2, один вид дрейфа и в основном большие изменения (медиана величины событий 0,59, доля событий с TV < 0,1 — 6,7 %); набор

Э2 — классов 62, три вида дрейфа, медиана 0,24, доля малых событий 20,9 %. Это количественный аргумент в пользу автоматического формирования наборов: выводы о детекторах, полученные на ручном наборе, не переносятся на пространство сценариев в целом.

4.9 Э7. Реальные данные с внедрённым дрейфом

Вопрос эксперимента: переносятся ли выводы о детекторах с синтетического семейства на реальные данные с внедрённым дрейфом. Для наборов Bananas и Phishing рассматриваются три вида дрейфа и две величины ($m \in \{0,1;0,3\}$): три постепенных события шириной 1000, $\pi_0=0,45$, $n=20\,000$. Синтетический двойник — те же параметры в семействе hyperplane_gauss той же размерности; всего сценариев — 24, из них с предупреждением W4 — 7. Recall по видам дрейфа приведён в таблице 4.19.

Таблица 4.19 — Recall по видам дрейфа: столбцы 2–4 — реальные данные, столбцы 5–7 — синтетические двойники

Детектор	real	virtual	prior	real	virtual	prior
PageHinkley-up	0,72	0,14	0,28	0,89	0,00	0,17
FHDDM	0,75	0,22	0,33	0,50	0,00	0,00
HDDM_W	0,83	0,47	0,42	0,33	0,00	0,00
PageHinkley	0,89	0,56	0,50	0,83	0,06	0,39
HDDM_A	0,53	0,11	0,28	0,67	0,00	0,17
ADWIN	0,69	0,17	0,33	0,78	0,14	0,28
DDM	0,56	0,06	0,19	0,97	0,00	0,25
KSWIN	0,36	0,11	0,22	0,00	0,00	0,00
EDDM	0,47	0,17	0,17	0,86	0,00	0,39

На реальных данных recall на виртуальном дрейфе в среднем на 0,20 выше, чем на синтетических двойниках (на дрейфе априорных вероятностей — на 0,12, на реальном — на 0,00). На реальных данных виртуальный дрейф меняет ошибку несовершенной модели и побочно меняет $P(y)$ (W4), поэтому «видимость» виртуального дрейфа — свойство пары «модель — данные», а не детектора. Согласие ранжирований по F1 на реальных данных и на синтетике не подтверждено: $\tau = 0,42$ ($p = 0,12$). Перенос выводов с синтетического семейства на реальные данные, таким образом, частичный.

4.10 Э8. Влияние на accuracy

Вопрос эксперимента: как обнаружение дрейфа сказывается на prequential accuracy базовой модели и согласуется ли ранжирование по accuracy с ранжированием по качеству обнаружения. Используются данные Э2 при $a=\text{reset}$: прирост accuracy относительно NoDrift на той же реализации потока и разрыв до Oracle (только для сценариев, у которых первое событие — реальный дрейф). Результаты приведены в таблице 4.20 и на рисунке 4.12.

Таблица 4.20 — Прирост accuracy, разрыв до Oracle и связь с F1 по сценариям

Метод	Δacc (среднее ± ст. откл.)	Разрыв до Oracle (real)	$\rho(\text{F1}, \Delta\text{acc})$	p (Холм)
PageHinkley-up	$0,011 \pm 0,033$	0,003	0,32	0,06
FHDDM	$0,010 \pm 0,034$	0,004	0,18	0,62
HDDM_W	$0,005 \pm 0,037$	0,008	0,05	1,00
PageHinkley	$-0,009 \pm 0,039$	0,021	0,06	1,00
HDDM_A	$0,012 \pm 0,033$	0,004	0,39	0,009
ADWIN	$0,006 \pm 0,035$	0,008	0,26	0,21
DDM	$0,012 \pm 0,032$	0,004	0,60	< 0,001
KSWIN	$0,004 \pm 0,022$	0,024	-0,02	1,00
EDDM	$0,007 \pm 0,029$	0,014	0,44	0,002
NoDrift	$0,000 \pm 0,000$	0,040	—	—
Periodic(2000)	$-0,014 \pm 0,037$	0,024	—	—
Oracle	$0,006 \pm 0,038$	0,000	—	—

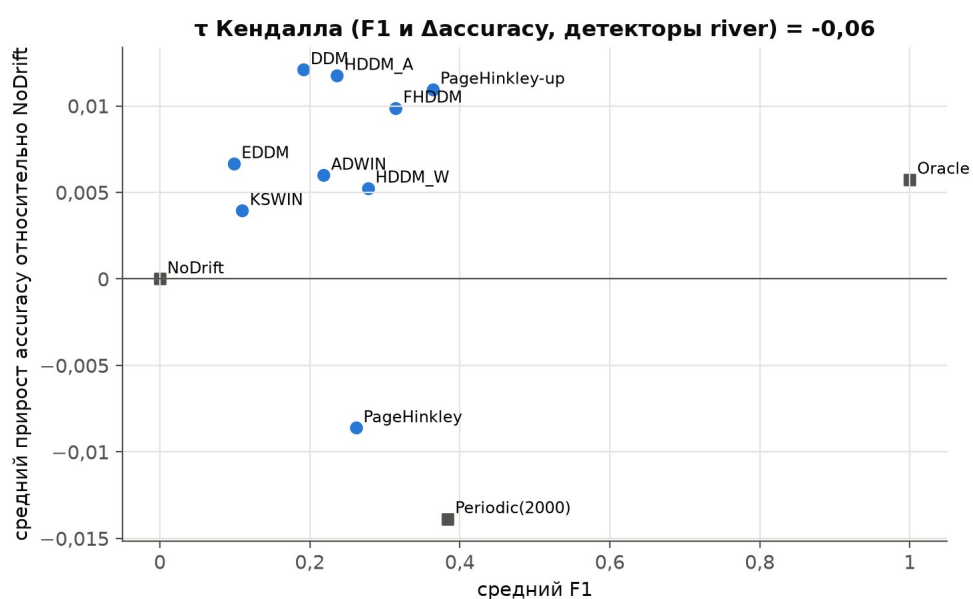


Рисунок 4.12 — Средний F1 и средний прирост accuracy детекторов и базовых линий

Ранжирования детекторов по F1 и по Δ ассигасу практически не связаны: $\tau = -0,06$. Наибольший средний прирост ассигасу — у DDM и HDDM_A (0,012), хуже всех — периодический сброс (-0,014) и двусторонний Page-Hinkley (-0,009). По сценариям связь F1 и Δ accuracy положительна и значима после поправки Холма только у DDM, EDDM, HDDM_A. Качество обнаружения и польза для модели — разные свойства детектора, и бенчмарк должен измерять оба.

4.11 Обсуждение и угрозы валидности

Результаты экспериментов согласуются с формальной моделью. Единая шкала величины оказалась содержательной: величина в TV — главный фактор сложности обнаружения, а доля затронутых признаков при фиксированной TV на качество не влияет. Виртуальный дрейф по потоку ошибок на синтетике не обнаруживается, что следует из утверждения 6. Автоматически сформированный набор дал выводы о детекторах, отличные от выводов на ручном наборе: качество ниже, порядок другой, а лучший по F1 метод — периодический сброс, не анализирующий данные. Эти результаты показывают, почему бенчмарк должен равномерно покрывать пространство сценариев и сопровождаться базовыми линиями.

Угрозы валидности:

- объём: в режиме FAST набор Э2 содержит 64 сценариев (число повторов — 3), поэтому мощность части проверок (анализ по одному фактору, правила отказа, MTD) невелика; [ЗАПУСТИТЬ FULL НА СЕРВЕРЕ и пересобирать записку: числа обновятся автоматически, утверждения выводов будут проверены повторно.]

- модель и параметры: базовая модель одна (гауссовский наивный байесовский классификатор), параметры детекторов — значения river по умолчанию; выводы относятся к этой конфигурации;

- семейство: синтетическое семейство линейно-гауссовское; перенос на реальные данные частичный, согласие ранжирований не подтверждено (Э7);

– протокол: окно допуска 2000 совпадает с периодом базовой линии Periodic, и при широких переходах окна покрывают большую часть потока ($\Theta 4$); поэтому вместе с recall приводится его превышение над случайным уровнем, рассчитанным в пуассоновском приближении;

– множественные сравнения: поправка Холма применяется внутри каждого семейства проверок, но не ко всем проверкам главы одновременно.

4.12 Выводы по главе 4

1) Генератор реализует заявленные свойства сценариев: отклонения фактической величины от заданной согласуются с выборочной погрешностью для всех видов дрейфа, кривые перехода следуют теоретическим, доля класса и шум воспроизводятся, потоки побитово воспроизводимы. Классические генераторы дают неконтролируемую величину смены концепта от 0,09 до 1,0.

2) Последовательность Соболя — наиболее равномерный и расширяемый план; из способов учёта ограничений условные домены сохраняют бюджет и баланс плана без атомов на границе, отклонение сохраняет равномерность на допустимой области ценой потери бюджета, проекция создаёт атом на границе.

3) На автоматически сформированном наборе средний F1 детекторов river — от 0,10 до 0,36, различия значимы, а периодический сброс получает F1 0,38, выше любого детектора, но худший прирост ассигасу.

4) Главный фактор сложности — величина дрейфа на единой шкале; виртуальный дрейф по потоку ошибок на синтетике не обнаруживается; шкала сложности λ работает для большинства детекторов, но требует ограничить рост ширины перехода.

5) Ручной набор классических потоков завышает качество и даёт несогласованное ранжирование; перенос выводов на реальные данные частичный; ранжирования по F1 и по ассигасу не связаны.

6) Все выводы главы (25 утверждений) проверены кодом и выполняются на результатах режима FAST.

ЗАКЛЮЧЕНИЕ

В работе разработан фреймворк scendrift для генерации бенчмарков систем выявления концептуального дрейфа с автоматизированным формированием и параметризацией сценариев. Цель работы достигнута, поставленные задачи решены.

1) Проанализированы виды концептуального дрейфа, методы его обнаружения, существующие средства генерации потоков и оценки детекторов (MOA и CD-MOA, scikit-multiflow, river, CapyMOA, stream-learn, Frouros, Menelaus, генератор Петижана–Уэбба) и исследовательские бенчмарки. Выявлены пять пробелов: нет общей семантики величины дрейфа, сценарий не является проверяемым объектом, наборы сценариев формируются вручную, перебором или случайно, генерация, метрики и статистика разнесены по разным инструментам, области отказа детекторов изучаются по одному параметру.

2) Построена формальная модель сценария дрейфа $S = (B, E, \Omega, s)$ с 14 ограничениями допустимости и 4 предупреждениями. Введена единая шкала величины дрейфа — расстояние полной вариации между совместными распределениями; для калиброванного семейства получены точные выражения величины реального, виртуального и априорного дрейфа, калибровка реального дрейфа с учётом дисбаланса и доли затронутых признаков и ограничения достижимости величины. Утверждения модели подтверждены методом Монте-Карло.

3) Спроектирована архитектура фреймворка из шести подпакетов и декларативный формат сценария (YAML, JSON Schema) с канонической и компактной формами, наследованием шаблонов и идентификацией по содержимому.

4) Реализованы генераторы потоков трёх групп семейств — калиброванного hyperplane_gauss, классических генераторов river и внедрения дрейфа в реальные данные — с автоматическим построением истинной разметки и фактической величины каждого события.

5) Реализован модуль автоматического формирования наборов сценариев: четыре плана эксперимента, три способа учёта ограничений (отклонение, проекция, условные домены), шаблоны, композиция из блоков, шкала сложности, манифест и проверка целостности. Доказано, что условные домены дают допустимые точки без атомов на границе.

6) Реализованы единый интерфейс девяти детекторов *river* и трёх базовых линий, векторная базовая модель, раннер с повторами, кэшем и параллельным запуском, метрики обнаружения, статистическое сравнение по Демшару и отчёты.

7) Проведено экспериментальное исследование. Генератор воспроизводит заданную величину дрейфа в пределах выборочной погрешности. На автоматически сформированном наборе средний F1 детекторов — от 0,10 до 0,36, периодический сброс без анализа данных получает 0,38, но худший прирост ассигасу. Главный фактор сложности — величина дрейфа на единой шкале; виртуальный дрейф по потоку ошибок на синтетических данных не обнаруживается; ручной набор классических потоков завышает качество и даёт иное ранжирование; ранжирования по F1 и по ассигасу не связаны.

Научная новизна работы состоит в формальной модели сценария с проверяемыми ограничениями, единой аналитически калиброванной шкале величины дрейфа для трёх видов изменения, методе формирования наборов сценариев по планам эксперимента с условными доменами и методике описания областей отказа детекторов с поправкой на случайный уровень обнаружения.

Практическая значимость работы: фреймворк позволяет выбирать детектор под характеристики прикладной задачи, проверять новые детекторы на воспроизводимых наборах сценариев и избегать ошибок протокола оценки, выявленных в работе (влияние сброса модели на двусторонние детекторы, «иллюзия прогресса» периодического сброса, расширение окон допуска при широких переходах).

Ограничения работы: калиброванное синтетическое семейство одно (бинарная классификация, линейная граница, гауссовы признаки); ограничение достижимости величины — свойство выбранной параметризации; базовая модель одна, а детекторы сравниваются с параметрами по умолчанию; окно допуска фиксировано; числа экспериментального исследования получены в режиме FAST. [ЗАПУСТИТЬ FULL НА СЕРВЕРЕ и пересобрать записку: числа обновятся автоматически, утверждения выводов будут проверены повторно.]

Направления развития: калиброванные семейства с нелинейной границей и многоклассовой разметкой; детекторы без учителя, отслеживающие распределение признаков, для которых виртуальный дрейф наблюдаем; подбор гиперпараметров детекторов под область пространства сценариев; окно допуска, зависящее от ширины перехода; уточнение шкалы сложности с упором на величину дрейфа.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. A survey on concept drift adaptation / J. Gama, I. Žliobaitė, A. Bifet [et al.] // ACM Computing Surveys. – 2014. – Vol. 46, no. 4. – Art. 44. – P. 1–37. – DOI 10.1145/2523813.
2. Learning under concept drift: a review / J. Lu, A. Liu, F. Dong [et al.] // IEEE Transactions on Knowledge and Data Engineering. – 2019. – Vol. 31, no. 12. – P. 2346–2363. – DOI 10.1109/TKDE.2018.2876857.
3. Barros, R. S. M. A large-scale comparison of concept drift detectors / R. S. M. Barros, S. G. T. C. Santos // Information Sciences. – 2018. – Vol. 451–452. – P. 348–370. – DOI 10.1016/j.ins.2018.04.014.
4. Bifet, A. Classifier concept drift detection and the illusion of progress / A. Bifet // Artificial Intelligence and Soft Computing : 16th International Conference, ICAISC 2017 : proceedings, part II. – Cham : Springer, 2017. – P. 715–725. – (Lecture Notes in Computer Science ; vol. 10246). – DOI 10.1007/978-3-319-59060-8_64.
5. Characterizing concept drift / G. I. Webb, R. Hyde, H. Cao [et al.] // Data Mining and Knowledge Discovery. – 2016. – Vol. 30, no. 4. – P. 964–994. – DOI 10.1007/s10618-015-0448-4.
6. Minku, L. L. The impact of diversity on online ensemble learning in the presence of concept drift / L. L. Minku, A. P. White, X. Yao // IEEE Transactions on Knowledge and Data Engineering. – 2010. – Vol. 22, no. 5. – P. 730–742. – DOI 10.1109/TKDE.2009.156.
7. Petitjean, F. ConceptDriftGenerator : MOA extension for generating categorical data streams with drift of a given magnitude / F. Petitjean, G. I. Webb // GitHub. – URL: <https://github.com/fpetitjean/ConceptDriftGenerator> (дата обращения: 06.10.2026). – Текст : электронный.
8. Carrera, D. Generating high-dimensional datastreams for change detection / D. Carrera, G. Boracchi // Big Data Research. – 2018. – Vol. 11. – P. 11–21. [ПРОВЕРИТЬ]

9. Aguiar, G. J. A comprehensive analysis of concept drift locality in data streams / G. J. Aguiar, A. Cano // Knowledge-Based Systems. – 2024. – Vol. 289. – Art. 111535. [ПРОВЕРИТЬ]
10. A framework for evaluating and benchmarking concept drift detection methods / V. Cerqueira, H. M. Gomes, M. Heyden [et al.] // arXiv. – 2026. – arXiv:2606.07789. – URL: <https://arxiv.org/abs/2606.07789> (дата обращения: 06.10.2026). – Текст : электронный. [ПРОВЕРИТЬ]
11. River: machine learning for streaming data in Python / J. Montiel, M. Halford, S. M. Mastelini [et al.] // Journal of Machine Learning Research. – 2021. – Vol. 22, no. 110. – P. 1–8.
12. CD-MOA: change detection framework for massive online analysis / A. Bifet, J. Read, B. Pfahringer [et al.] // Advances in Intelligent Data Analysis XII : 12th International Symposium, IDA 2013 : proceedings. – Berlin ; Heidelberg : Springer, 2013. – P. 92–103. – (Lecture Notes in Computer Science ; vol. 8207). – DOI 10.1007/978-3-642-41398-8_9.
13. Demšar, J. Statistical comparisons of classifiers over multiple data sets / J. Demšar // Journal of Machine Learning Research. – 2006. – Vol. 7. – P. 1–30.
14. SciPy 1.0: fundamental algorithms for scientific computing in Python / P. Virtanen, R. Gommers, T. E. Oliphant [et al.] // Nature Methods. – 2020. – Vol. 17, no. 3. – P. 261–272. – DOI 10.1038/s41592-019-0686-2.
15. Machine learning for data streams : with practical examples in MOA / A. Bifet, R. Gavaldà, G. Holmes, B. Pfahringer. – Cambridge, MA : MIT Press, 2018. – 288 p. – (Adaptive Computation and Machine Learning). – ISBN 978-0-262-03779-2. – DOI 10.7551/mitpress/10654.001.0001.
16. Žliobaitė, I. Learning under concept drift: an overview / I. Žliobaitė // arXiv. – 2010. – arXiv:1010.4784. – URL: <https://arxiv.org/abs/1010.4784> (дата обращения: 06.10.2026). – Текст : электронный.
17. Harries, M. Splice-2 comparative evaluation: electricity pricing : technical report / M. Harries. – Sydney : The University of New South Wales, 1999. [ПРОВЕРИТЬ]

18. Hinder, F. One or two things we know about concept drift — a survey on monitoring in evolving environments. Part A: detecting concept drift / F. Hinder, V. Vaquet, B. Hammer // *Frontiers in Artificial Intelligence*. – 2024. – Vol. 7. – Art. 1330257. – DOI 10.3389/frai.2024.1330257.

19. A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams / D. Lukats, O. Zielinski, A. Hahn, F. Stahl // *International Journal of Data Science and Analytics*. – 2025. – Vol. 19, no. 1. – P. 1–31. – DOI 10.1007/s41060-024-00620-y.

20. A comparative study on concept drift detectors / P. M. Gonçalves Jr., S. G. T. de Carvalho Santos, R. S. M. Barros, D. C. L. Vieira // *Expert Systems with Applications*. – 2014. – Vol. 41, no. 18. – P. 8144–8156. – DOI 10.1016/j.eswa.2014.07.019.

21. A unifying view on dataset shift in classification / J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez [et al.] // *Pattern Recognition*. – 2012. – Vol. 45, no. 1. – P. 521–530. – DOI 10.1016/j.patcog.2011.06.019.

22. Learning with drift detection / J. Gama, P. Medas, G. Castillo, P. Rodrigues // *Advances in Artificial Intelligence – SBIA 2004 : 17th Brazilian Symposium on Artificial Intelligence : proceedings*. – Berlin ; Heidelberg : Springer, 2004. – P. 286–295. – (Lecture Notes in Computer Science ; vol. 3171). – DOI 10.1007/978-3-540-28645-5_29.

23. Early drift detection method / M. Baena-García, J. del Campo-Ávila, R. Fidalgo [et al.] // *Fourth International Workshop on Knowledge Discovery from Data Streams (IWKDDs 2006)*, held with ECML PKDD 2006. – Berlin, 2006. – P. 77–86. [ПРИБЛИЖИТЬ]

24. Online and non-parametric drift detection methods based on Hoeffding's bounds / I. Frías-Blanco, J. del Campo-Ávila, G. Ramos-Jiménez [et al.] // *IEEE Transactions on Knowledge and Data Engineering*. – 2015. – Vol. 27, no. 3. – P. 810–823. – DOI 10.1109/TKDE.2014.2345382.

25. Pesaranghader, A. Fast Hoeffding drift detection method for evolving data streams / A. Pesaranghader, H. L. Viktor // *Machine Learning and Knowledge*

Discovery in Databases : European Conference, ECML PKDD 2016 : proceedings, part II. – Cham : Springer, 2016. – P. 96–111. – (Lecture Notes in Computer Science ; vol. 9852). – DOI 10.1007/978-3-319-46227-1_7.

26. Bifet, A. Learning from time-changing data with adaptive windowing / A. Bifet, R. Gavaldà // Proceedings of the 2007 SIAM International Conference on Data Mining. – Philadelphia : SIAM, 2007. – P. 443–448. – DOI 10.1137/1.9781611972771.42.

27. Raab, C. Reactive soft prototype computing for concept drift streams / C. Raab, M. Heusinger, F.-M. Schleif // Neurocomputing. – 2020. – Vol. 416. – P. 340–351. – DOI 10.1016/j.neucom.2019.11.111.

28. Page, E. S. Continuous inspection schemes / E. S. Page // Biometrika. – 1954. – Vol. 41, no. 1/2. – P. 100–115. – DOI 10.1093/biomet/41.1-2.100.

29. MOA: Massive Online Analysis / A. Bifet, G. Holmes, R. Kirkby, B. Pfahringer // Journal of Machine Learning Research. – 2010. – Vol. 11. – P. 1601–1604.

30. Street, W. N. A streaming ensemble algorithm (SEA) for large-scale classification / W. N. Street, Y. Kim // KDD '01 : Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. – New York : ACM, 2001. – P. 377–382. – DOI 10.1145/502512.502568.

31. Agrawal, R. Database mining: a performance perspective / R. Agrawal, T. Imielinski, A. Swami // IEEE Transactions on Knowledge and Data Engineering. – 1993. – Vol. 5, no. 6. – P. 914–925. – DOI 10.1109/69.250074.

32. Schlimmer, J. C. Incremental learning from noisy data / J. C. Schlimmer, R. H. Granger Jr. // Machine Learning. – 1986. – Vol. 1, no. 3. – P. 317–354. – DOI 10.1007/BF00116895.

33. Hulten, G. Mining time-changing data streams / G. Hulten, L. Spencer, P. Domingos // KDD '01 : Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. – New York : ACM, 2001. – P. 97–106. – DOI 10.1145/502512.502529.

34. Scikit-Multiflow: a multi-output streaming framework / J. Montiel, J. Read, A. Bifet, T. Abdesslem // *Journal of Machine Learning Research*. – 2018. – Vol. 19, no. 72. – P. 1–5.
35. CapyMOA: efficient machine learning for data streams in Python / H. M. Gomes, A. Lee, N. Gunasekara [et al.] // *arXiv*. – 2025. – arXiv:2502.07432. – URL: <https://arxiv.org/abs/2502.07432> (дата обращения: 06.10.2026). – Текст : электронный. [ПРОВЕРИТЬ]
36. Ksieniewicz, P. stream-learn — open-source Python library for difficult data stream batch analysis / P. Ksieniewicz, P. Zyblewski // *Neurocomputing*. – 2022. – Vol. 478. – P. 11–21. – DOI 10.1016/j.neucom.2021.10.120.
37. Céspedes Sisniega, J. Frouros: an open-source Python library for drift detection in machine learning systems / J. Céspedes Sisniega, Á. López García // *SoftwareX*. – 2024. – Vol. 26. – Art. 101733. – DOI 10.1016/j.softx.2024.101733.
38. Menelaus: online and batch-based concept and data drift detection algorithms / The MITRE Corporation // *GitHub*. – Version 0.2.0. – 2022. – URL: <https://github.com/mitre/menelaus> (дата обращения: 06.10.2026). – Текст : электронный.
39. The impact of data difficulty factors on classification of imbalanced and concept drifting data streams / D. Brzezinski, L. L. Minku, T. Pewinski [et al.] // *Knowledge and Information Systems*. – 2021. – Vol. 63. – P. 1429–1469. [ПРОВЕРИТЬ]
40. Žliobaitė, I. Controlled permutations for testing adaptive learning models / I. Žliobaitė // *Knowledge and Information Systems*. – 2014. – Vol. 39, no. 3. – P. 565–578.
41. Change detection in multivariate datastreams: likelihood and detectability loss / C. Alippi, G. Boracchi, D. Carrera, M. Roveri // *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence (IJCAI 2016)*. – 2016. – P. 1368–1374.
42. A synthetic benchmark to explore limitations of localized drift detections / F. Giobergia, E. Pastor, L. de Alfaro, E. Baralis // *arXiv*. – 2024. –

arXiv:2408.14687. – URL: <https://arxiv.org/abs/2408.14687> (дата обращения: 06.10.2026). – Текст : электронный. [ПРОВЕРИТЬ]

43. Rabanser, S. Failing loudly: an empirical study of methods for detecting dataset shift / S. Rabanser, S. Günnemann, Z. C. Lipton // *Advances in Neural Information Processing Systems* 32 (NeurIPS 2019). – 2019. [ПРОВЕРИТЬ]

44. Challenges in benchmarking stream learning algorithms with real-world data / V. M. A. Souza, D. M. dos Reis, A. G. Maletzke, G. E. A. P. A. Batista // *Data Mining and Knowledge Discovery*. – 2020. – Vol. 34, no. 6. – P. 1805–1858. – DOI 10.1007/s10618-020-00698-5.

45. McKay, M. D. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code / M. D. McKay, R. J. Beckman, W. J. Conover // *Technometrics*. – 1979. – Vol. 21, no. 2. – P. 239–245. – DOI 10.1080/00401706.1979.10489755.

46. Соболев, И. М. О распределении точек в кубе и приближенном вычислении интегралов / И. М. Соболев // *Журнал вычислительной математики и математической физики*. – 1967. – Т. 7, № 4. – С. 784–802.

47. Instance spaces for machine learning classification / M. A. Muñoz, L. Villanova, D. Baatar, K. Smith-Miles // *Machine Learning*. – 2018. – Vol. 107. – P. 109–147. [ПРОВЕРИТЬ]

48. Goemans, M. X. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming / M. X. Goemans, D. P. Williamson // *Journal of the ACM*. – 1995. – Vol. 42, no. 6. – P. 1115–1145. – DOI 10.1145/227683.227684.

49. Owen, D. B. Tables for computing bivariate normal probabilities / D. B. Owen // *The Annals of Mathematical Statistics*. – 1956. – Vol. 27, no. 4. – P. 1075–1090.

50. Plackett, R. L. A reduction formula for normal multivariate integrals / R. L. Plackett // *Biometrika*. – 1954. – Vol. 41, no. 3/4. – P. 351–360.

51. Gama, J. On evaluating stream learning algorithms / J. Gama, R. Sebastião, P. P. Rodrigues // *Machine Learning*. – 2013. – Vol. 90, no. 3. – P. 317–346. – DOI 10.1007/s10994-012-5320-9.
52. Hickernell, F. J. A generalized discrepancy and quadrature error bound / F. J. Hickernell // *Mathematics of Computation*. – 1998. – Vol. 67, no. 221. – P. 299–322. [ПРОВЕРИТЬ]
53. Friedman, M. The use of ranks to avoid the assumption of normality implicit in the analysis of variance / M. Friedman // *Journal of the American Statistical Association*. – 1937. – Vol. 32, no. 200. – P. 675–701. – DOI 10.1080/01621459.1937.10503522.
54. Nemenyi, P. B. Distribution-free multiple comparisons : PhD thesis / P. B. Nemenyi. – Princeton : Princeton University, 1963.
55. Wilcoxon, F. Individual comparisons by ranking methods / F. Wilcoxon // *Biometrics Bulletin*. – 1945. – Vol. 1, no. 6. – P. 80–83. – DOI 10.2307/3001968.
56. Holm, S. A simple sequentially rejective multiple test procedure / S. Holm // *Scandinavian Journal of Statistics*. – 1979. – Vol. 6, no. 2. – P. 65–70.
57. Kendall, M. G. A new measure of rank correlation / M. G. Kendall // *Biometrika*. – 1938. – Vol. 30, no. 1/2. – P. 81–93. – DOI 10.1093/biomet/30.1-2.81.
58. Scikit-learn: machine learning in Python / F. Pedregosa, G. Varoquaux, A. Gramfort [et al.] // *Journal of Machine Learning Research*. – 2011. – Vol. 12. – P. 2825–2830.
59. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления : межгосударственный стандарт : издание официальное : введен в действие в качестве национального стандарта Российской Федерации Приказом Федерального агентства по техническому регулированию и метрологии от 24 октября 2017 г. № 1494-ст :

взамен ГОСТ 7.32–2001 : дата введения 2018-07-01. – Москва : Стандартинформ, 2017. – Текст : непосредственный.

60. ГОСТ Р 7.0.100–2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления : национальный стандарт Российской Федерации : издание официальное : утвержден и введен в действие Приказом Федерального агентства по техническому регулированию и метрологии от 3 декабря 2018 г. № 1050-ст : дата введения 2019-07-01. – Москва : Стандартинформ, 2018. – Текст : непосредственный.

ПРИЛОЖЕНИЕ А

(справочное)

Каталог ограничений допустимости сценария

Каталог ограничений (C1–C14) и предупреждений (W1–W4) строится из кода фреймворка (модуль `scendrift.scenario.report`) при сборке пояснительной записки. Ограничения уровня «событие» проверяются при создании объекта, остальные — валидатором с учётом возможностей семейства генератора (подраздел 2.5).

Таблица А.1 — Ограничения допустимости и предупреждения

Код	Уровень	Условие
C1	событие	$\phi = \text{sudden} \Leftrightarrow \ell = 0$; при $\phi \in \{\text{gradual}, \text{incremental}\} \ell \geq 2$; $\text{onset} \geq 0$
C2	семейство	обычное событие: $m \in (0, 1]$ задана для калиброванного семейства и не задана для некалиброванного (там m оценивается по данным)
C3	событие	для повторяющегося события (p задано) k и m не задаются
C4	сценарий	$\text{onset}_i \geq W$: дрейф не начинается во время разогрева
C5	сценарий	$\text{end}_K + \Delta \leq n$: после последнего перехода есть окно допуска
C6	сценарий	$\text{onset}_{\{k+1\}} \geq \text{end}_k + \Delta$: следы событий $[\text{onset}, \text{end} + \Delta)$ не пересекаются
C7	сценарий	$p_k \leq k - 1$ (концепт уже существует), и возврат меняет распределение: $c_{\{p_k\}} \neq c_{\{k-1\}}$
C8	семейство	вид k , форма ϕ , возврат и ключи <code>family_params</code> поддерживаются семейством f
C9	семейство	число затронутых признаков $[\alpha \cdot d] \geq 2$ (для $k \in \{\text{real}, \text{virtual}\}$)
C10	семейство	real: $m \leq m_{\max}(q, \pi_0, \pi_k)$ — величина достижима поворотом внутри A
C11	семейство	virtual: $m \leq 0,99$ ($TV < 1$ при конечном сдвиге)
C12	семейство	prior: новая доля $P(y=1)$ лежит в $[0,01; 0,99]$
C13	семейство	семейство f зарегистрировано в реестре
C14	сценарий	$W + \Delta \leq n$: протокол оценки помещается в поток
W1	предупреждение	real: $m \cdot (1 - 2\eta) < 0,02$ — дрейф почти не наблюдаем на фоне шума меток
W2	предупреждение	virtual: $P(y X)$ не меняется, детекторы по ошибке могут его не видеть
W3	предупреждение	real при $\pi_k \neq \pi_0$: вместе с $P(y X)$ меняется и $P(X)$ (перевзвешивание классов)
W4	предупреждение	реальные данные: событие real или virtual меняет и $P(y)$ более чем на 0,02

ПРИЛОЖЕНИЕ Б

(справочное)

Примеры описания сценариев и конфигурации набора

Пример Б.1 — сценарий в канонической форме: один внезапный реальный дрейф величины 0,25 в середине потока.

```
schema_version: "1.0"
name: sudden_real_single
stream:
  family: hyperplane_gauss
  n_samples: 20000
  n_features: 10
  minority_share: 0.5
  label_noise: 0.0
  concept_seed: 0
events:
  - {position: 10000, form: sudden, kind: real, magnitude: 0.25,
    affected_share: 1.0}
evaluation:
  base_model: gaussian_nb
  on_detection: reset
  warmup: 1000
  acceptance_window: 2000
seed: 1
```

Пример Б.2 — сценарий в компактной форме: инкрементальные дрейфы со случайным расписанием и частичным охватом признаков (величина ограничена сверху ограничением C10).

```
schema_version: "1.0"
name: incremental_random_partial
stream: {n_samples: 40000, n_features: 15, minority_share: 0.5, concept_seed: 5}
drifts:
  schedule: {mode: random, count: 4, min_gap: 500}
  defaults: {form: incremental, kind: real, width: 3000, magnitude: 0.1,
    affected_share: 0.3}
  overrides:
    - {index: 3, shape: sigmoid}
seed: 3
```

Пример Б.3 — наследование профиля каталога с переопределением величины дрейфа.

```
extends: catalog/medium
drifts: {defaults: {magnitude: 0.3}}
```

Пример Б.4 — конфигурация набора сценариев основного бенчмарка Э2 (режим FAST).

```

name: e2_fast
version: 1.0.0
description: Основной бенчмарк Э2 (FAST) – план Соболя по 11 параметрам
template:
  extends: catalog/base
design: {method: sobol, n: 64, seed: 2027}
constraints: adapt

```

Пример Б.5 — формирование набора, сохранение с манифестом и проверка целостности на языке Python.

```

import yaml
from scndrift.formation import SuiteConfig, build_suite, load_suite

config = SuiteConfig.model_validate(
    yaml.safe_load(open("configs/suites/e2_fast.yaml", encoding="utf-8")))
suite = build_suite(config)          # план Соболя -> сценарии -> условные
домены
suite.save("results/suites/e2_fast") # manifest.yaml и scenarios/*.yaml
assert load_suite("results/suites/e2_fast").suite_id == suite.suite_id

```

ПРИЛОЖЕНИЕ В

(обязательное)

Распределение работ между исполнителями

Работа выполнена коллективом из двух исполнителей: Будаев Константин Владимирович отвечал за сценарии и данные, Гарифзянов Тимур Русланович — за оценку и анализ. Ответственный исполнитель указан также в строке документации каждого модуля фреймворка и в заголовке каждого раздела исполняемого ноутбука. Распределение работ приведено в таблице В.1.

Таблица В.1 — Распределение работ между исполнителями

Часть работы	Будаев К. В.	Гарифзянов Т. Р.	Совместно
Введение	—	—	целиком
Глава 1	1.1, 1.2: потоки, дрейф, таксономия	1.3, 1.4: детекторы, средства оценки, метрики	1.5, 1.6
Глава 2	2.1–2.5, 2.7: сценарий, шкала величины, пространство, ограничения, формирование наборов	2.6: протокол оценки и метрики	2.8
Глава 3	3.3–3.5: формат сценария, генерация, формирование наборов	3.6: детекторы, раннер, метрики, статистика, отчёты	3.1, 3.2, 3.7, 3.8
Глава 4	4.2 (Э1), 4.3, 4.9 (Э7)	4.4 (Э2), 4.5 (Э3), 4.7 (Э5), 4.8 (Э6), 4.10 (Э8)	4.1, 4.6 (Э4), 4.11, 4.12
Заключение	—	—	целиком
Модули фреймворка	scenario, generators, formation	detectors, evaluation, reporting	interfaces, сборка ноутбука и записки

ПРИЛОЖЕНИЕ Г

(справочное)

Структура программного комплекса и воспроизведение результатов

Программный комплекс размещён в репозитории и включает:

- VKR_Budaev_Garifzyanov.ipynb — исполняемый ноутбук, основной артефакт работы: текст и работающий фреймворк, выполненные сверху вниз;
- scendrift/ — пакет фреймворка (подпакеты scenario, formation, generators, detectors, evaluation, reporting);
- notebook/ — исходные тексты разделов ноутбука и список литературы references.yaml;
- configs/ — примеры сценариев, конфигурации наборов и JSON Schema сценария;
- tests/ — модульные тесты pytest;
- tools/ — сборка ноутбука, схемы архитектуры, оформление списка литературы по ГОСТ Р 7.0.100–2018;
- results/ — результаты экспериментов в формате CSV и сохранённые наборы сценариев;
- docs/zapiska/ — исходные тексты и сборщик настоящей пояснительной записки.

Результаты воспроизводятся командами (из каталога vkr):

```
python -m pip install -r requirements.txt
python -m pytest                                # модульные тесты
python tools/build_notebook.py --execute         # ноутбук и все
эксперименты (FAST)
SCENDRIFT_MODE=FULL python tools/build_notebook.py --execute # полный объём
python docs/zapiska/build_docx.py                # пояснительная записка
(.docx и .pdf)
```

Сборщик записки подставляет в текст числа и таблицы из файлов results/*.csv, нумерует рисунки, таблицы, формулы и источники, оформляет документ по ГОСТ 7.32–2017 [59] и список источников по ГОСТ Р 7.0.100–2018 [60]; все параметры оформления заданы в файле docs/zapiska/format_config.py.